

COMP0222 Simultaneous Localisation and Mapping

Lecture 09: Feature Descriptors, Matching and Loop Closing

Simon Julier

s.julier@ucl.ac.uk

Sajad Saeedi

s.saeedi@ucl.ac.uk

Motivation

- In all the lectures so far, we've looked at SLAM as *an estimation problem*
- A sensor takes a snapshot of the environment around the robot



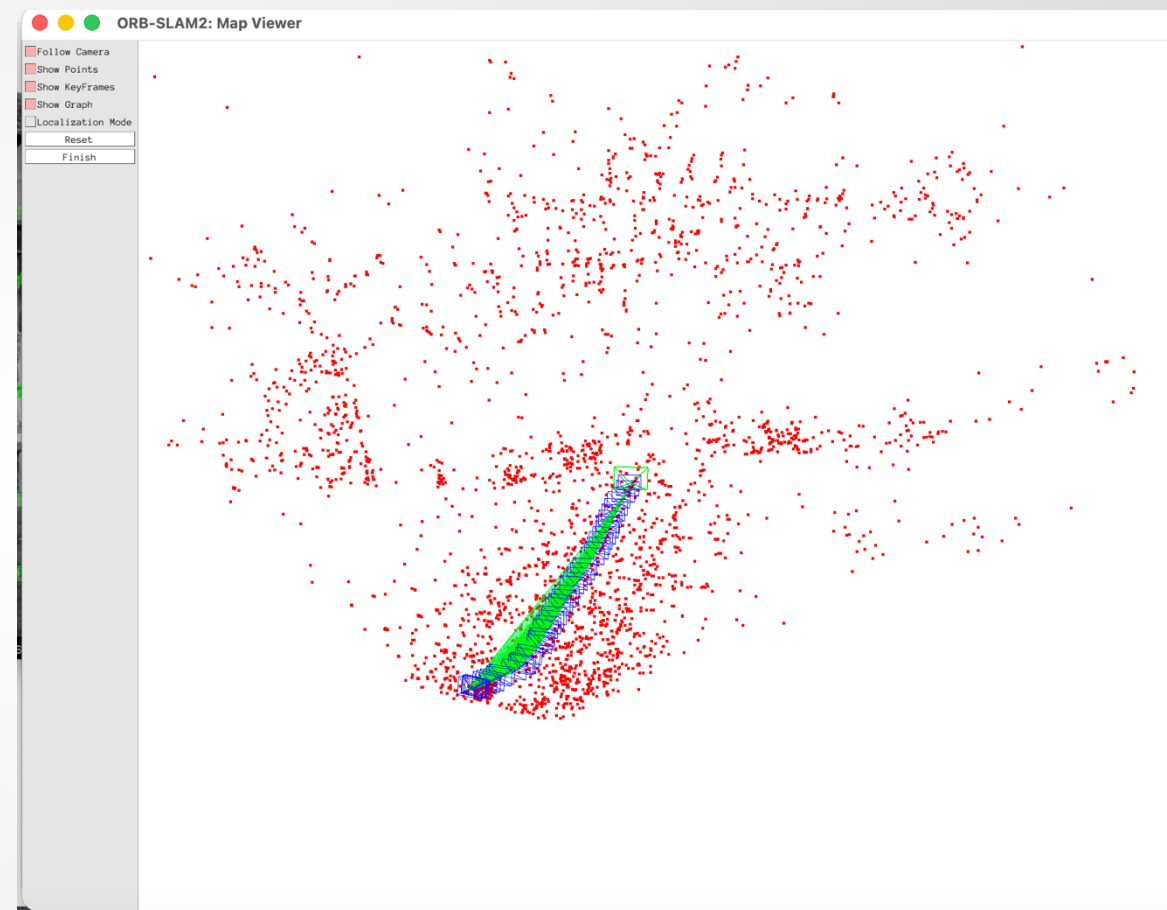
Motivation

- In all the lectures so far, we've looked at SLAM as an estimation problem
- A sensor takes a snapshot of the environment around the robot
- A set of features come from *somewhere*



Motivation

- In all the lectures so far, we've looked at SLAM as an estimation problem
- A sensor takes a snapshot of the environment around the robot
- A set of features come from *somewhere*
- We have *been told* how those features associate with landmarks in the map





Motivation

- In all the lectures so far, we've looked at SLAM as an estimation problem
- A sensor takes a snapshot of the environment around the robot
- A set of features come from *somewhere*
- We have *been told* how those features associate with landmarks in the map
- We can then just run our factor graph

Motivation

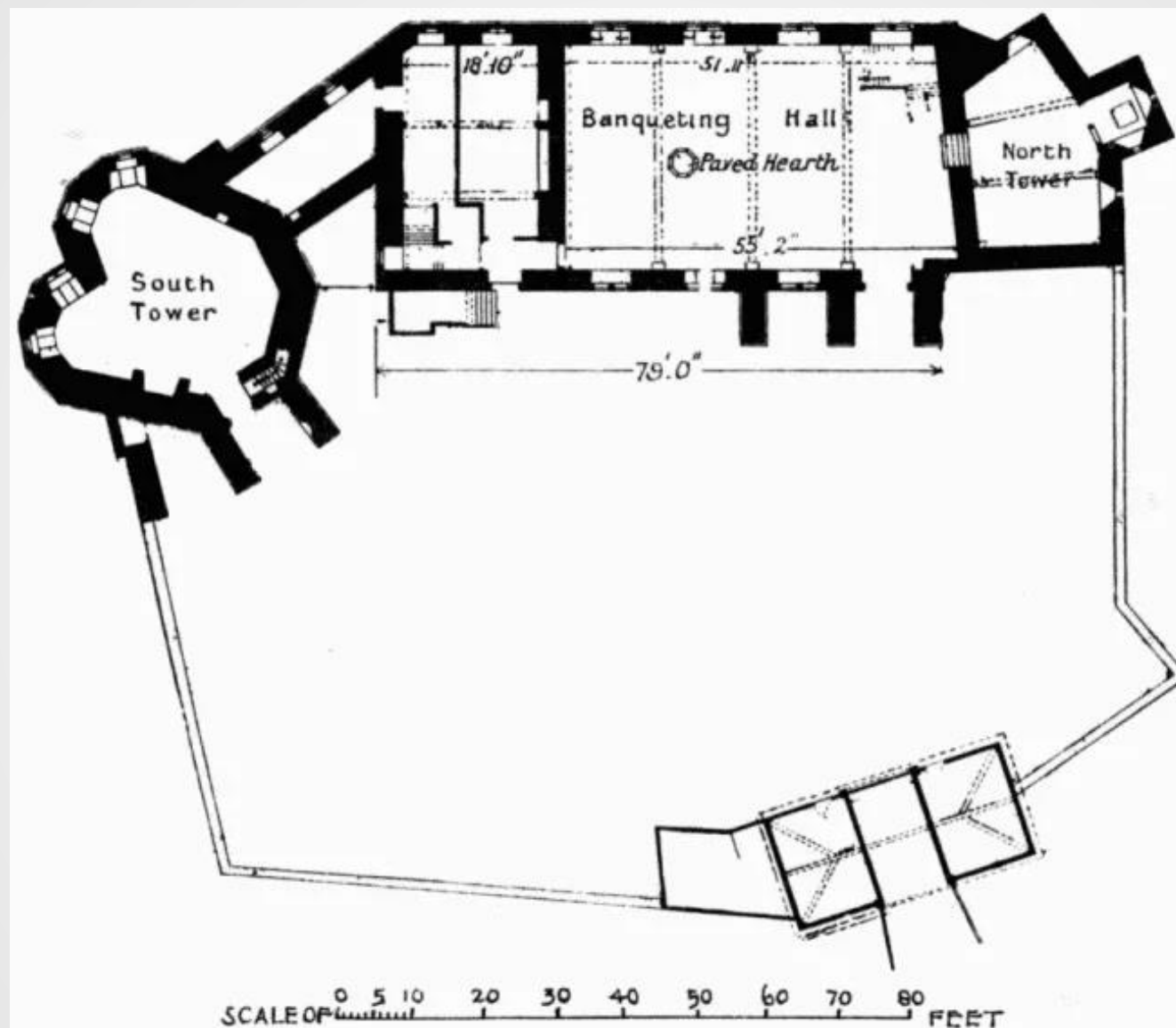
- However, in reality nobody magically tells you this stuff
- We have to work it out directly from the data
- In this lecture, we will look at the problems of feature extraction and data association
- We'll focus on camera-based SLAM
- We'll see there are *two* types of association problem: feature matching and place recognition

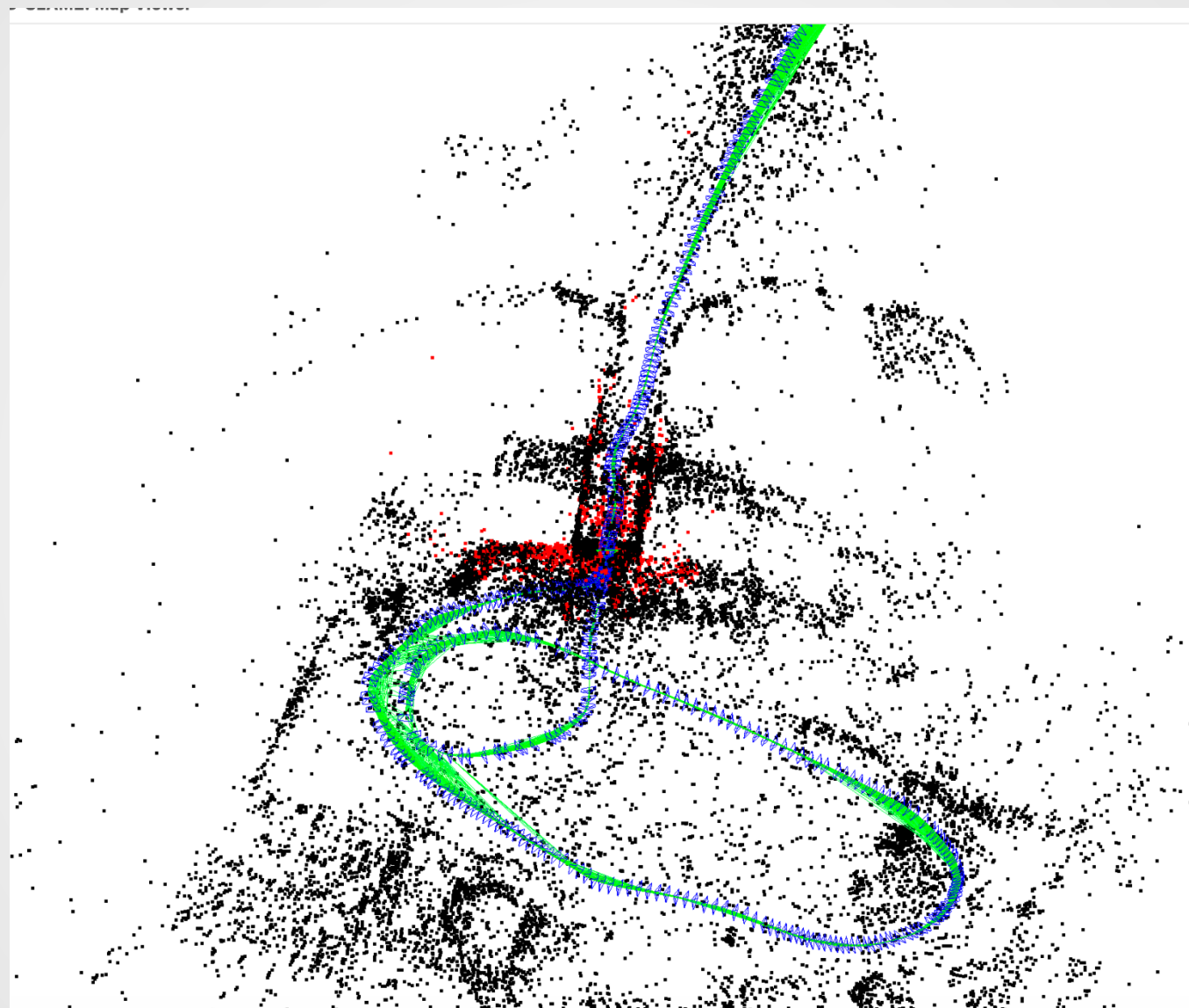
Frame Matching



Place Recognition







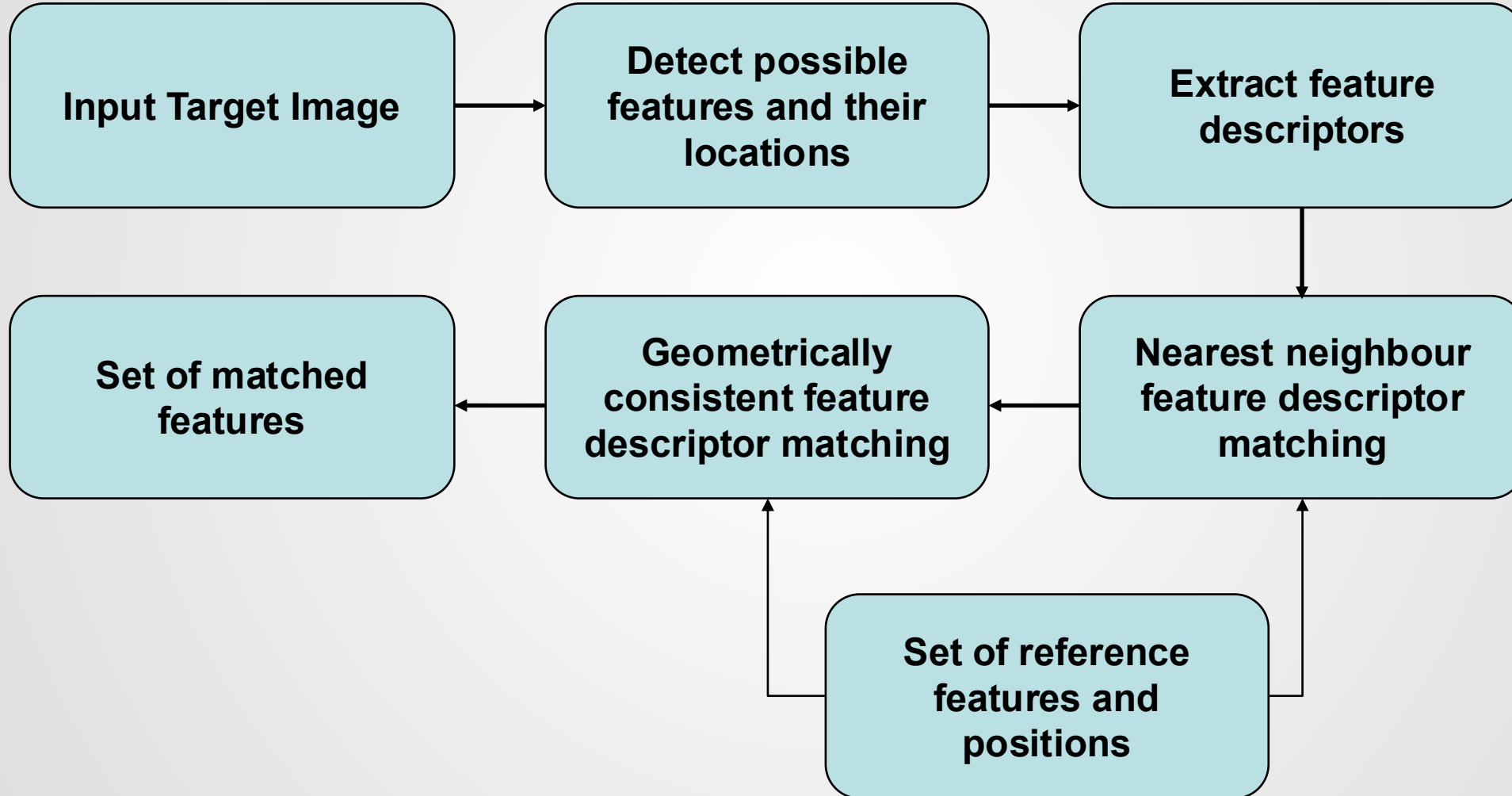
Structure

- **Frame-to-Frame Feature Matching**
- **Place Recognition and Loop Closing**

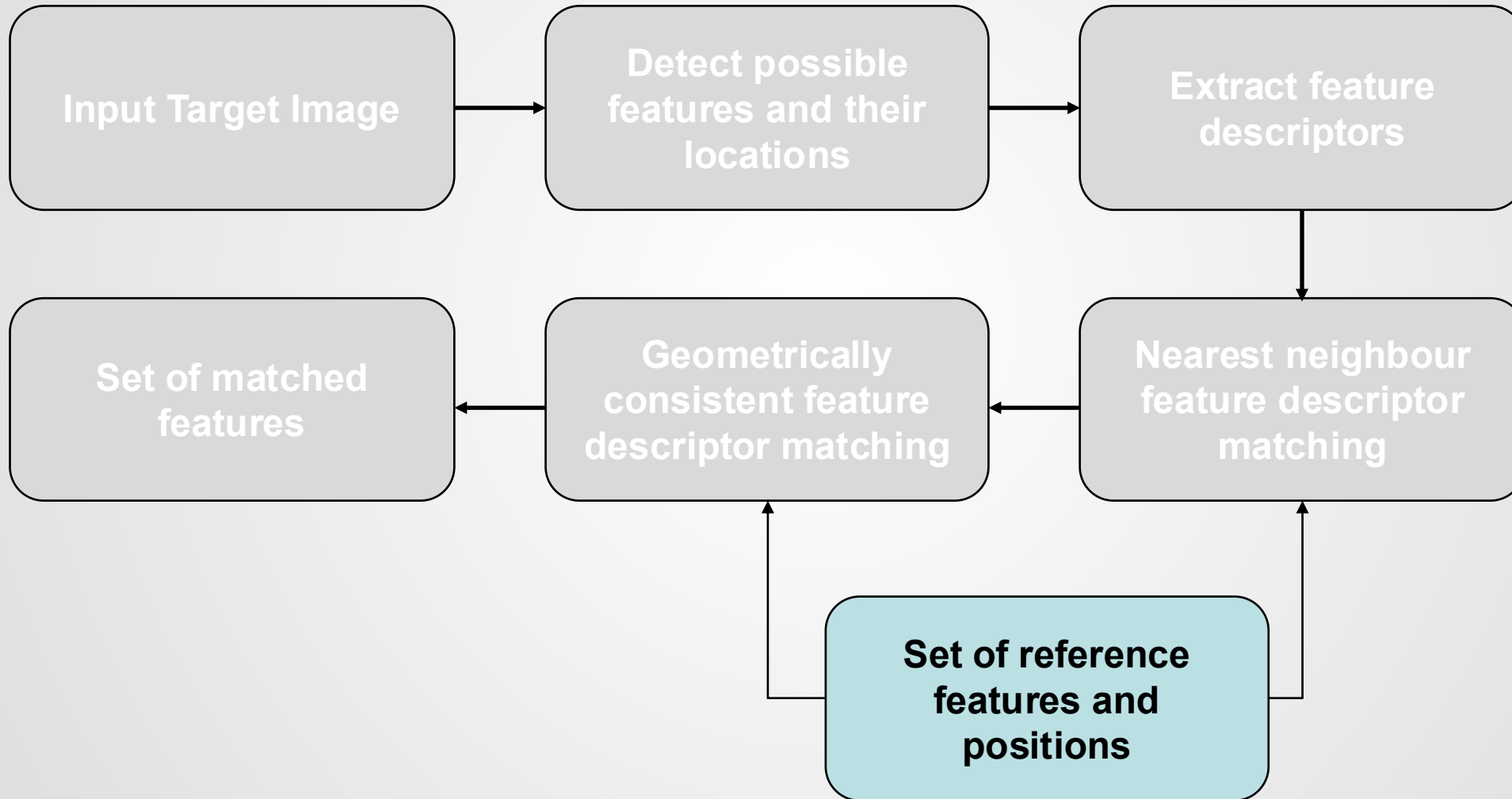
Frame-to-Frame Feature Matching

- Frame-to-Frame Feature Matching
- *Place Recognition and Loop Closing*

Overview of the Feature Matching Process



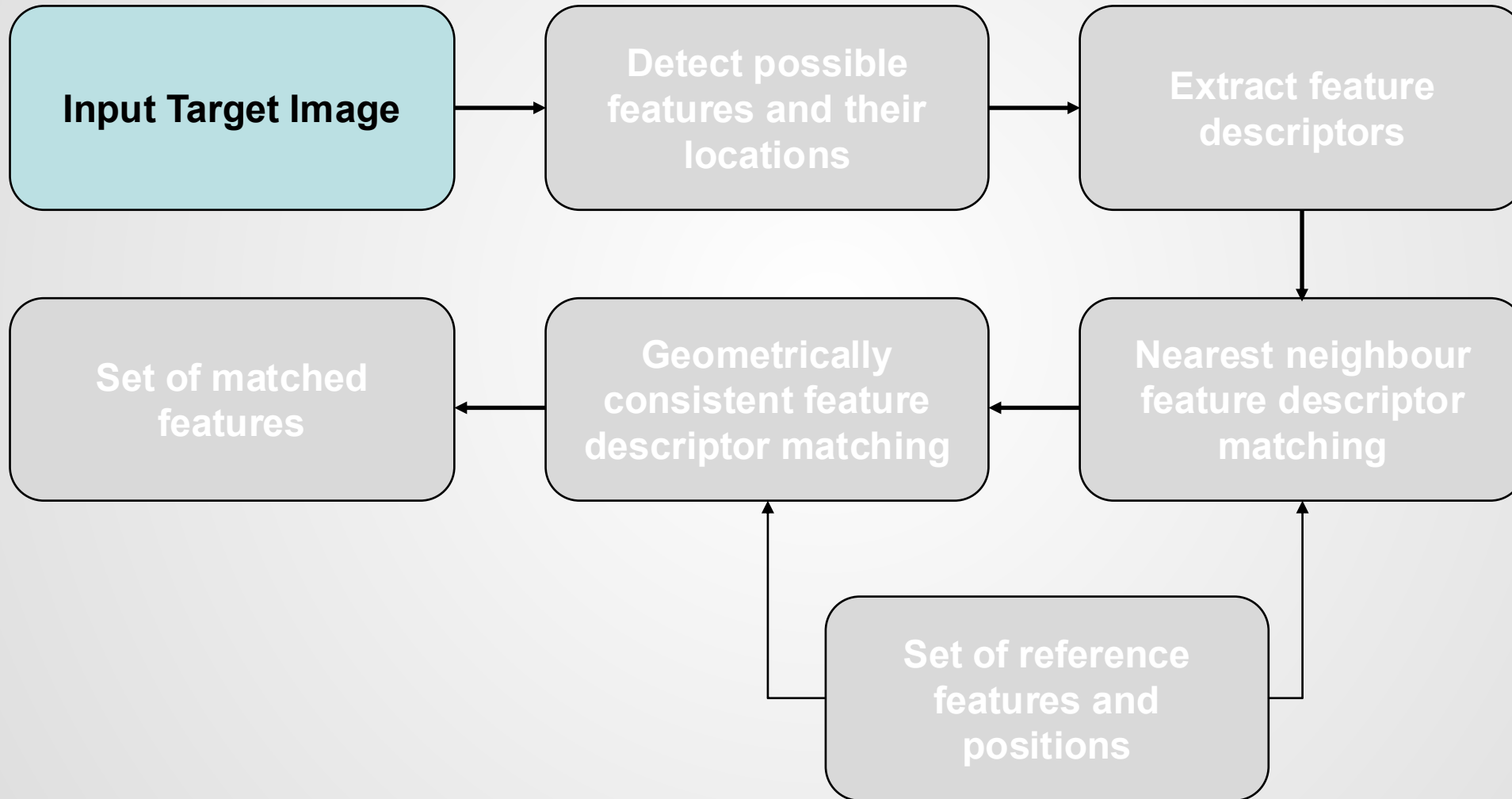
Overview of the Feature Matching Process



Set of Reference Features and Positions

- In a SLAM system there are typically *two* types of reference features and positions
- Features detected in previous images:
 - Typically used for initialization
 - Position is pixel coordinates in previous frames
- Map features:
 - Used to track with the map
 - Position is the landmark position estimate

Overview of the Feature Matching Process



Input Target Image



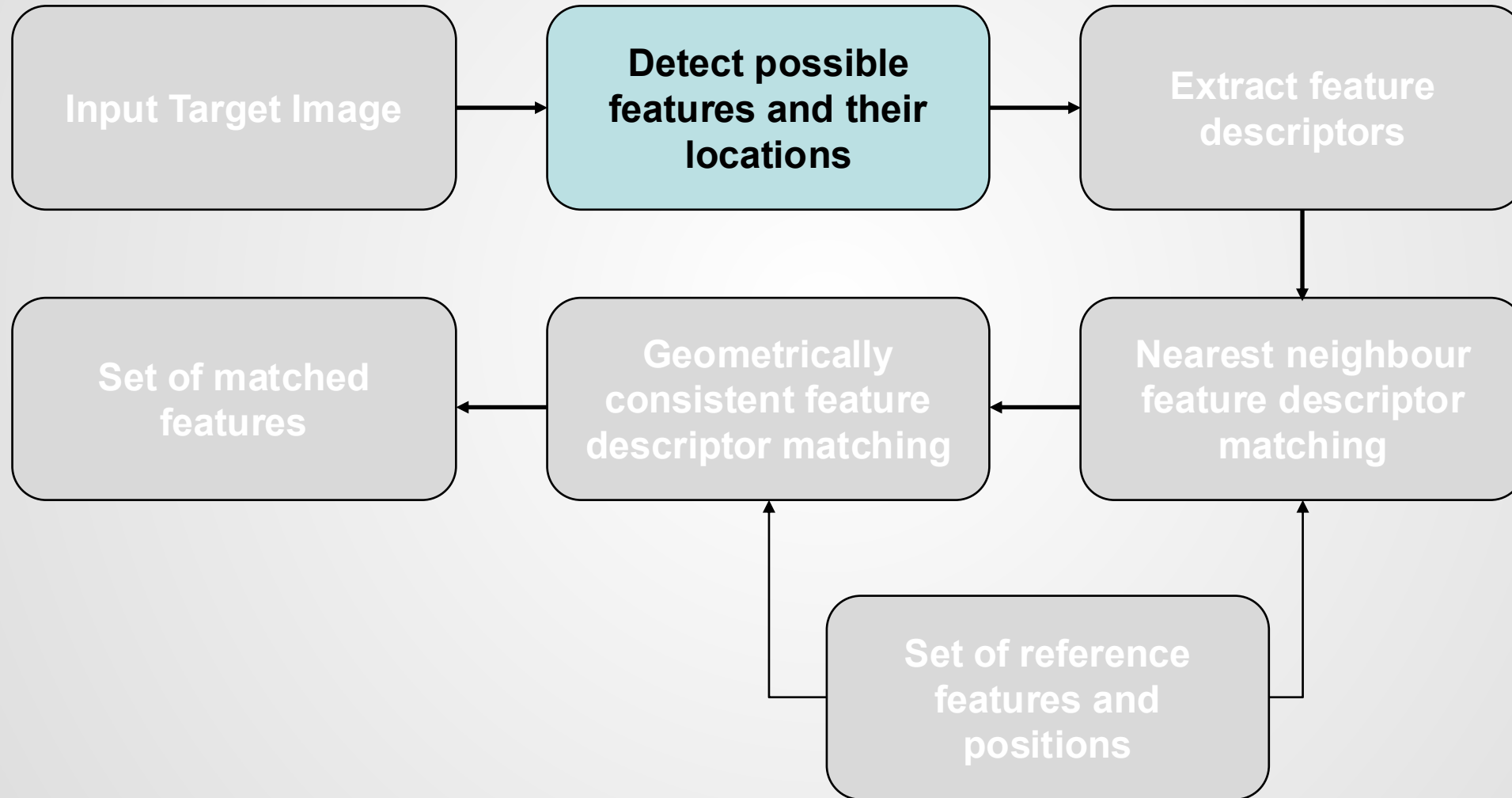
Input Image

- **Most feature detection algorithms only work in black and white**
- **Furthermore, their speed decreases with image size**
- **Therefore, input images are usually converted to black-and-white, blurred and sub-sampled**

Processed Input Image



Overview of the Feature Matching Process



Detect Possible Features and their Locations

- **We process the input image to find candidate locations where trackable features might be**
- **To do this, we have to remind ourselves of what features these features should have, and how we can set up a detector to find them**

Feature Properties and Detectors

- **The goal is to find features which are anchored to real-world, stationary objects in the environment**
- **These are usually visually distinct regions caused by things such as imperfections and cracks**
- **They rarely line up with what we would consider to be objects in a scene**



Feature Detectors

- The features can appear anywhere in an image
- Therefore, we have to run a detector over all of the pixels
- Detection algorithms need to be *fast*
- Therefore, most approaches are simple and potentially allow through a lot of false positives
- A lot of approaches are based on image gradients

Common Feature Detectors

- **Difference-of-Gaussians**
- **FAST corner detector**
- **Superpoint Interest Point Extractor**

Difference-of-Gaussians

- The difference-of-Gaussians approach is a way to robustly identify “real-world” features over different scales
- The key idea is to think in terms of the frequency of the signal in the image:
 - Noise is a very high frequency signal
 - Big, drawn out surfaces are very low signal
 - Corners, edges, etc., lie in a band pass in the middle
- This means that it’s a natural “blob detector”

Gaussian-Blurred Images

Single octave 1: Gaussian scale space

G k0
 $\sigma = 1.60$



G k1
 $\sigma = 2.02$



G k2
 $\sigma = 2.54$



G k3
 $\sigma = 3.20$



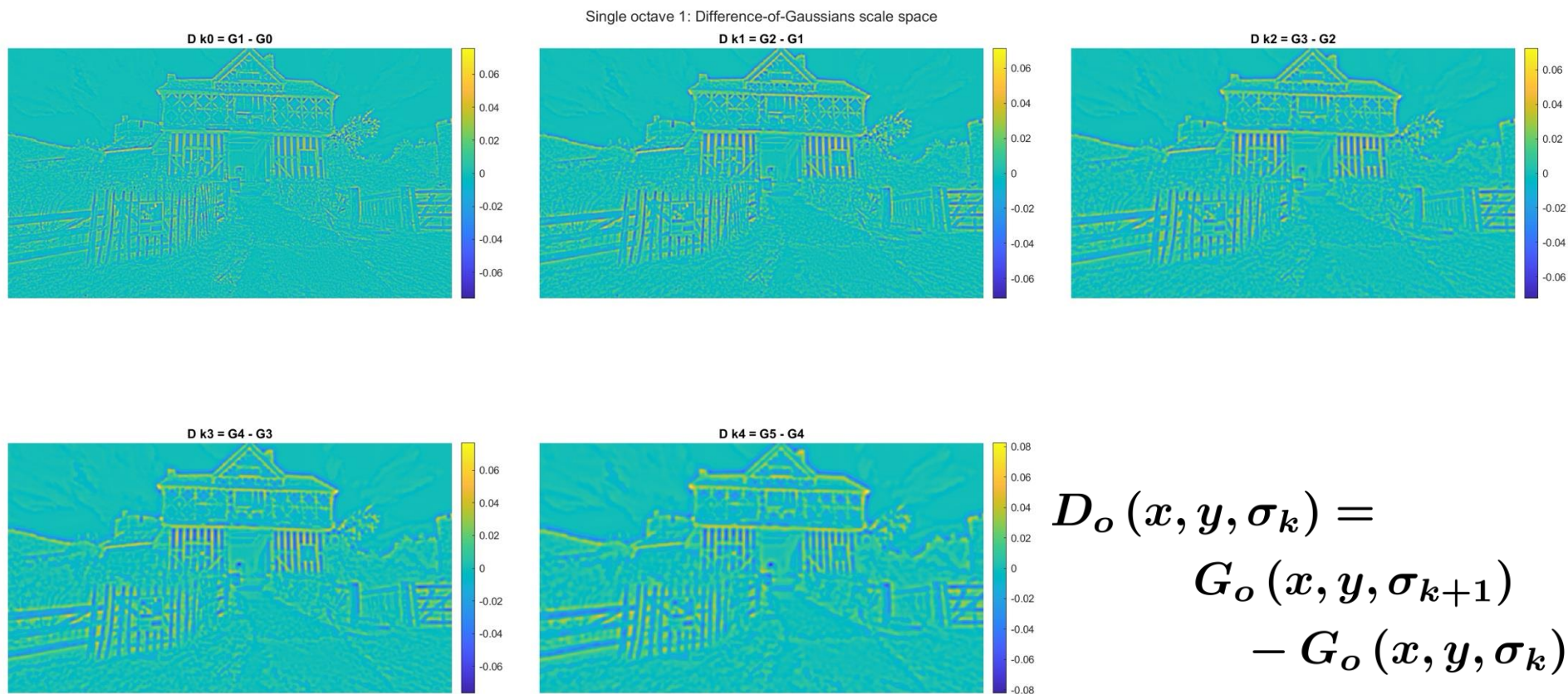
G k4
 $\sigma = 4.03$



G k5
 $\sigma = 5.08$

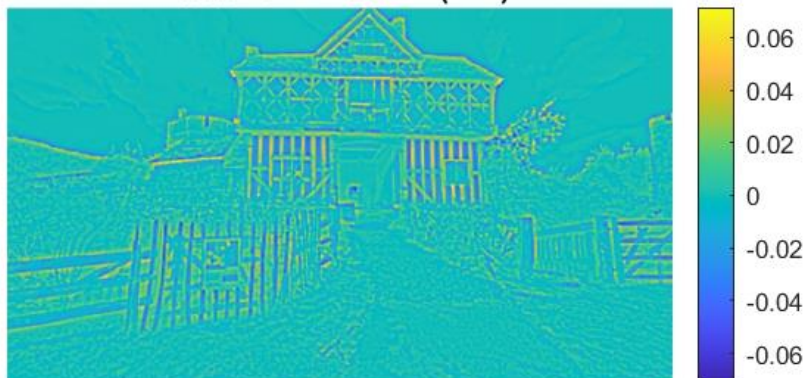


Difference-of-Gaussians

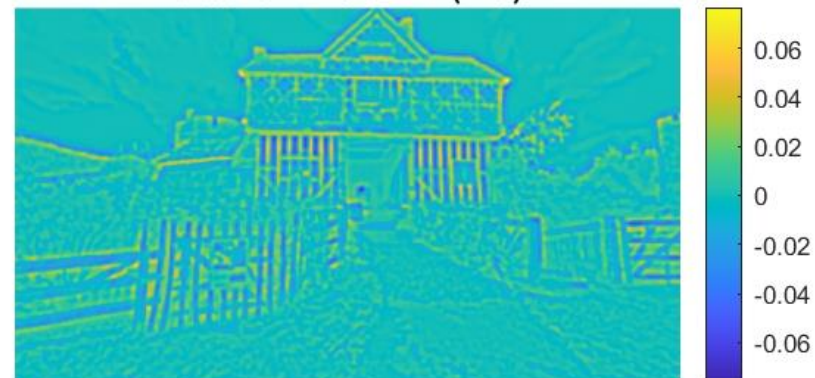


Extrema Points

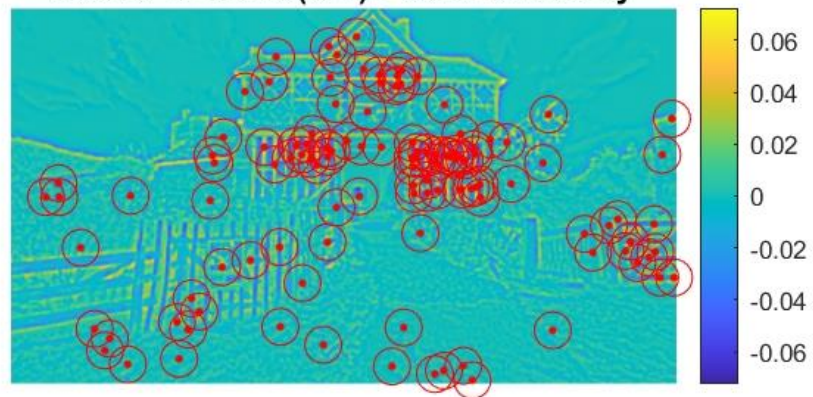
Octave 1: DoG $k-1$ ($k=1$)



Octave 1: DoG $k+1$ ($k=3$)



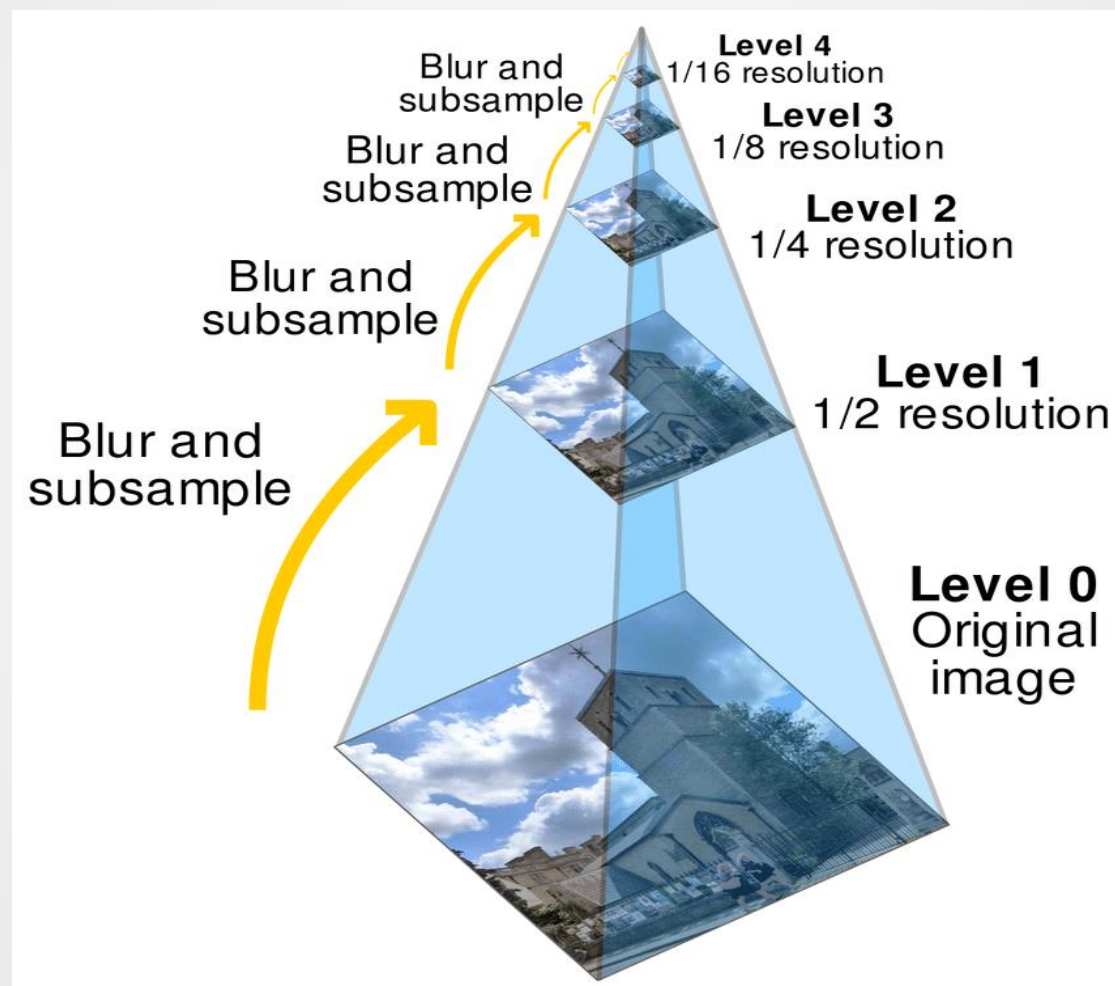
Octave 1: DoG k ($k=2$) + extrema overlay



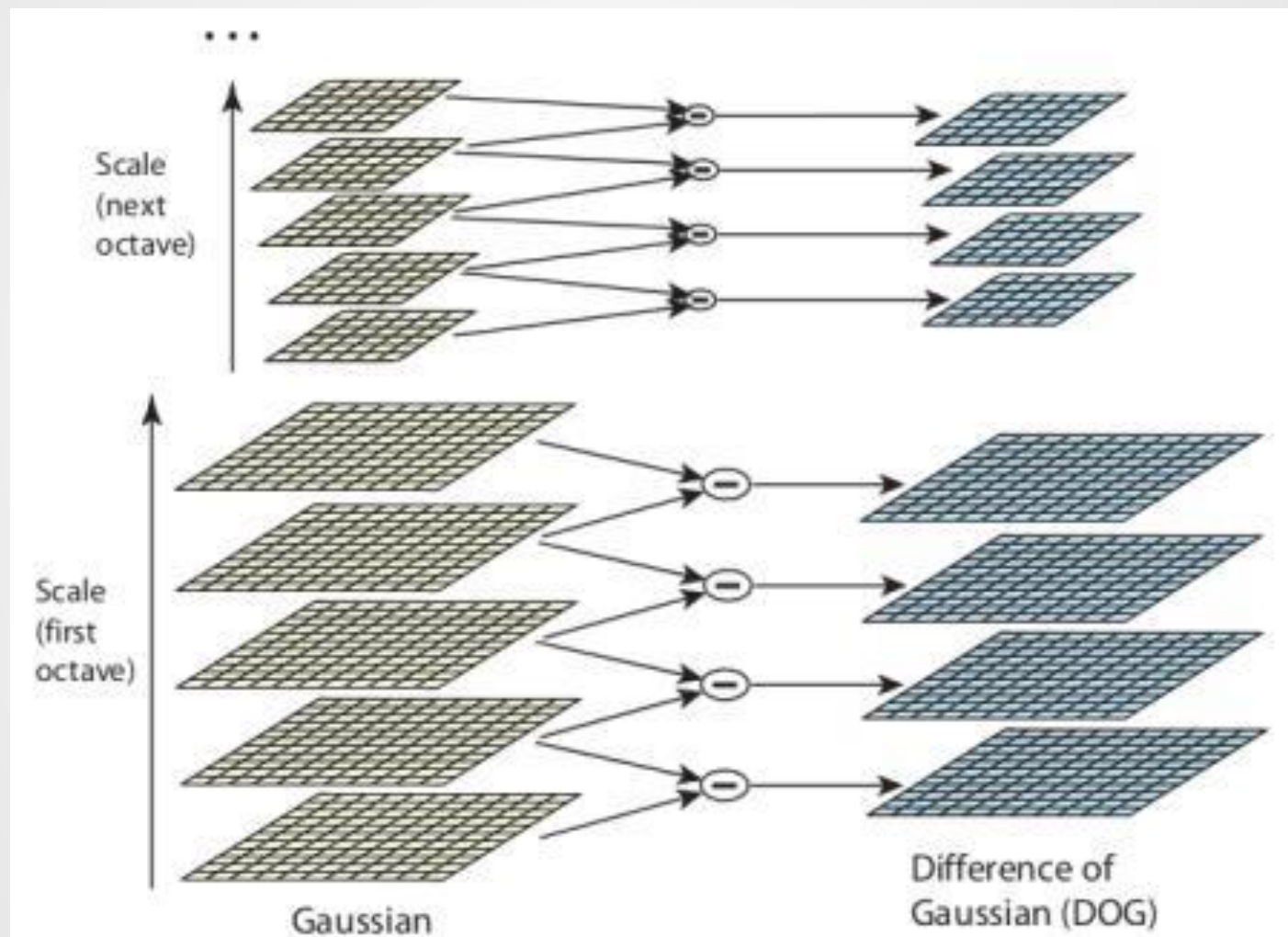
Octaves

- So far, we've seen how we have a way to generate points of interest at a single image resolution
- Algorithms such as SIFT enhance this by operating over multiple image resolutions by downsampling the image
- This helps to introduce invariance to scale changes

Image Octaves

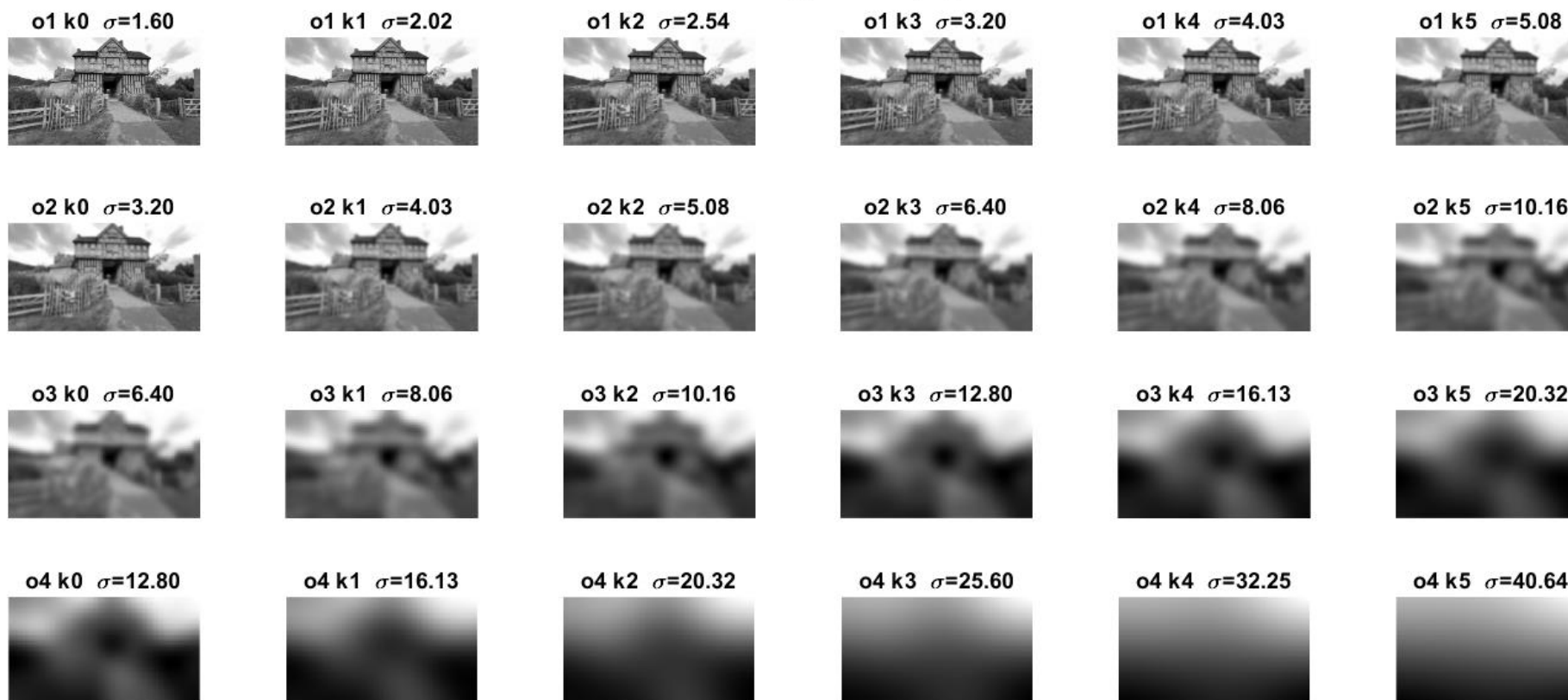


DoGs on Different Octaves



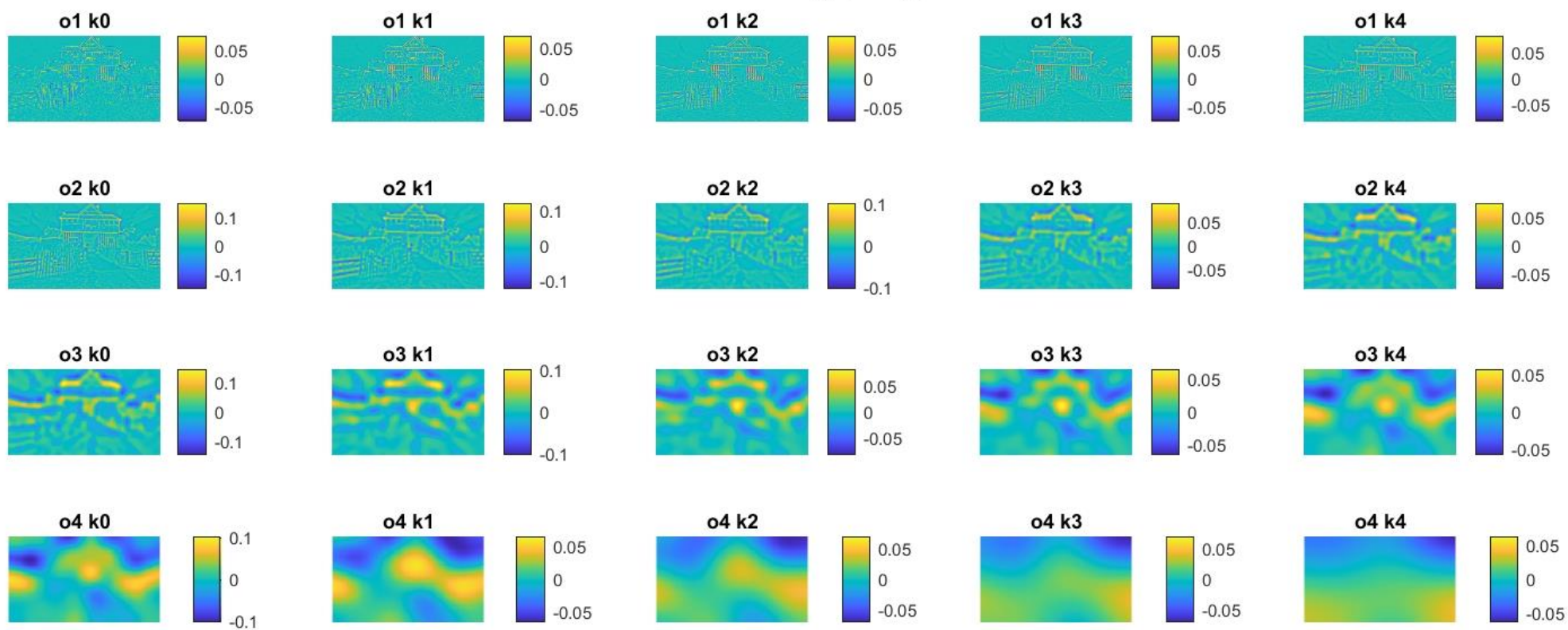
Gaussian Pyramid

Gaussian pyramid (G)



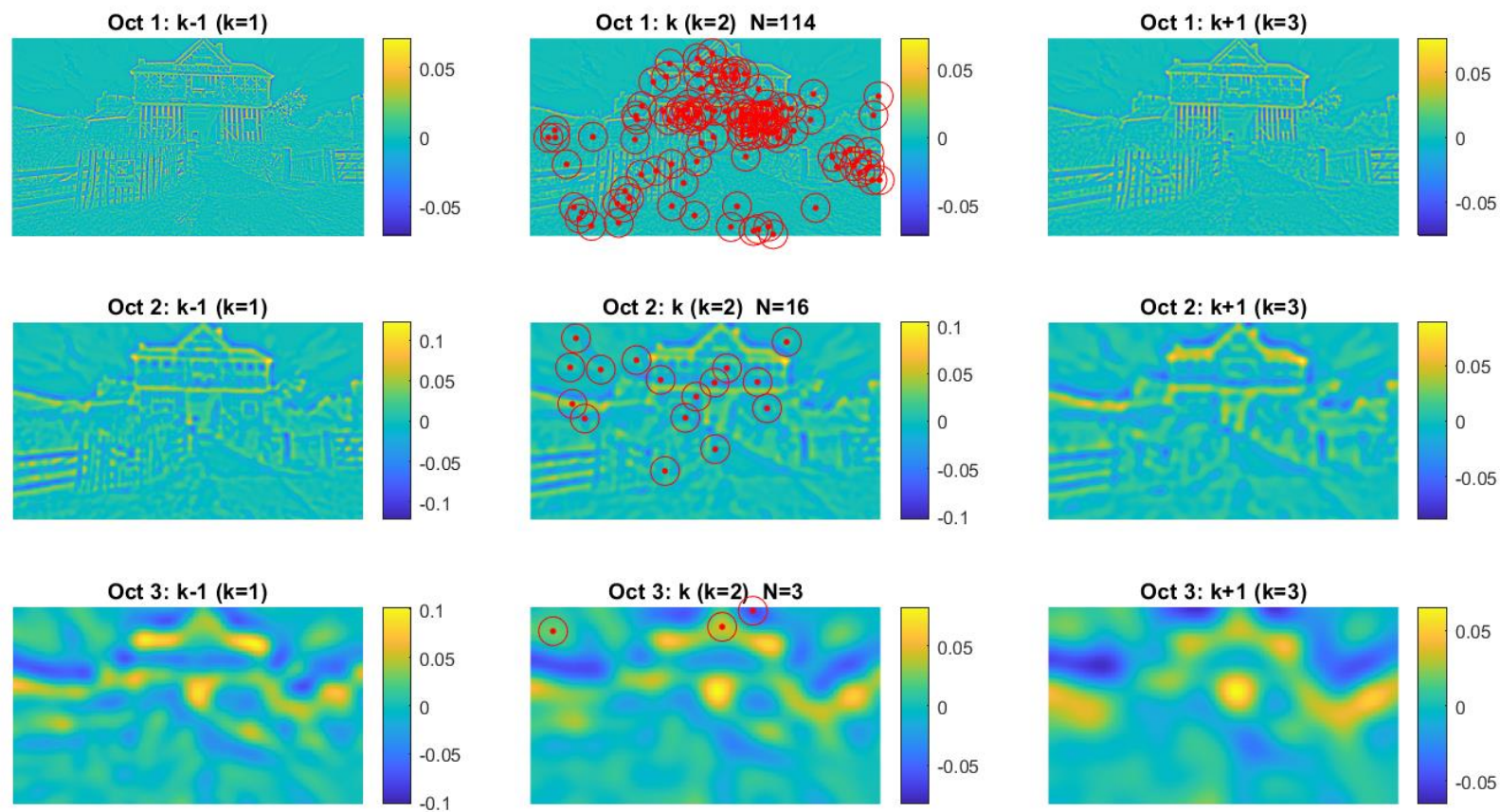
DoGs on Gaussian Pyramid

DoG pyramid ($D = G_{k+1} - G_k$)



Extracted Extrema Points

3D extrema test across octaves (threshold $|\text{DoG}| > 0.030$)



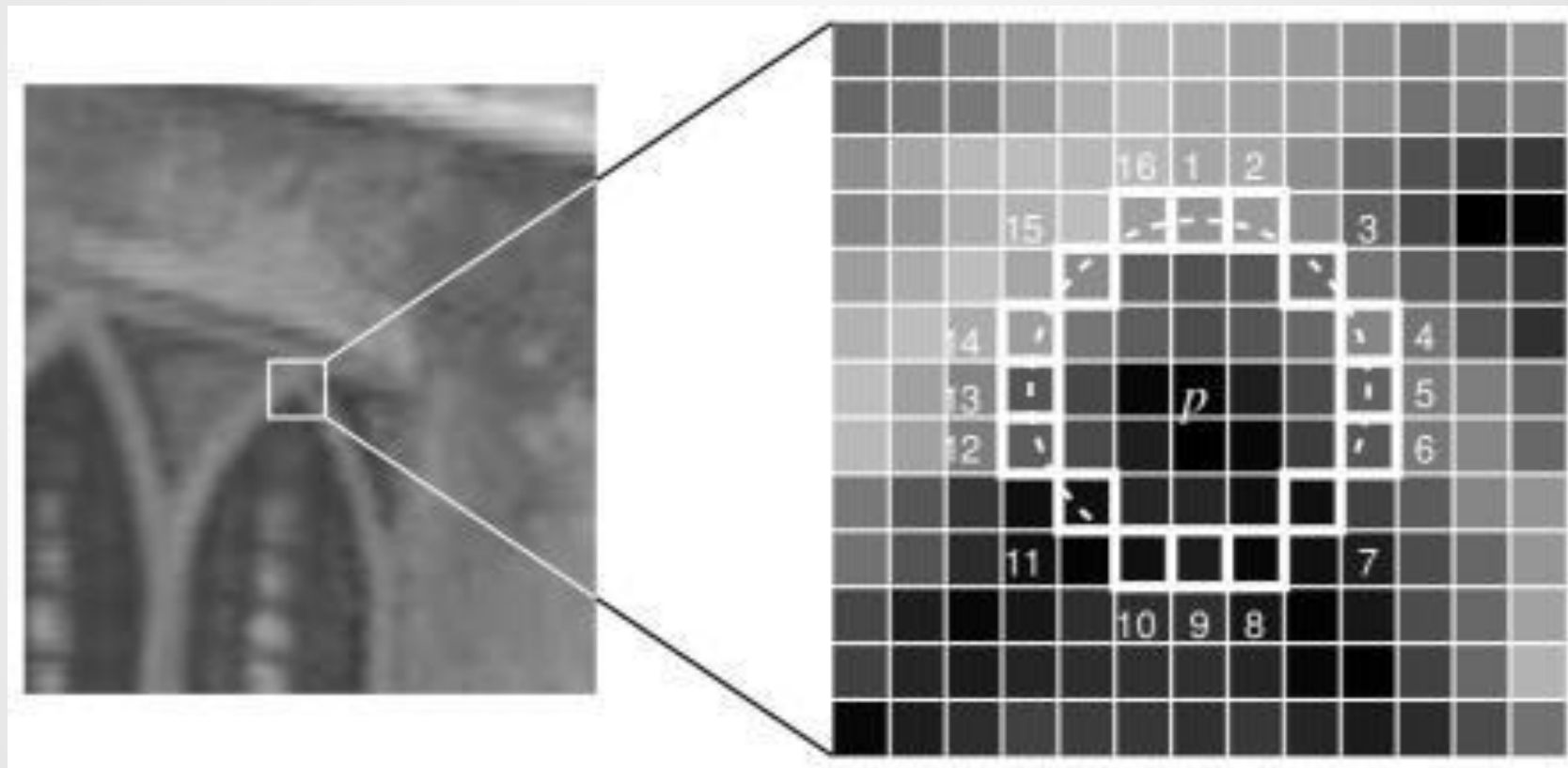
Using Difference-of-Gaussians to Extract Interest Points

- Get a set of points at a single octave:
- $\{(x_i, y_i, \sigma_i)\}_o$
- $\text{Keypoints} = \bigcup_o \{(x_i, y_i, \sigma_i)\}_o$

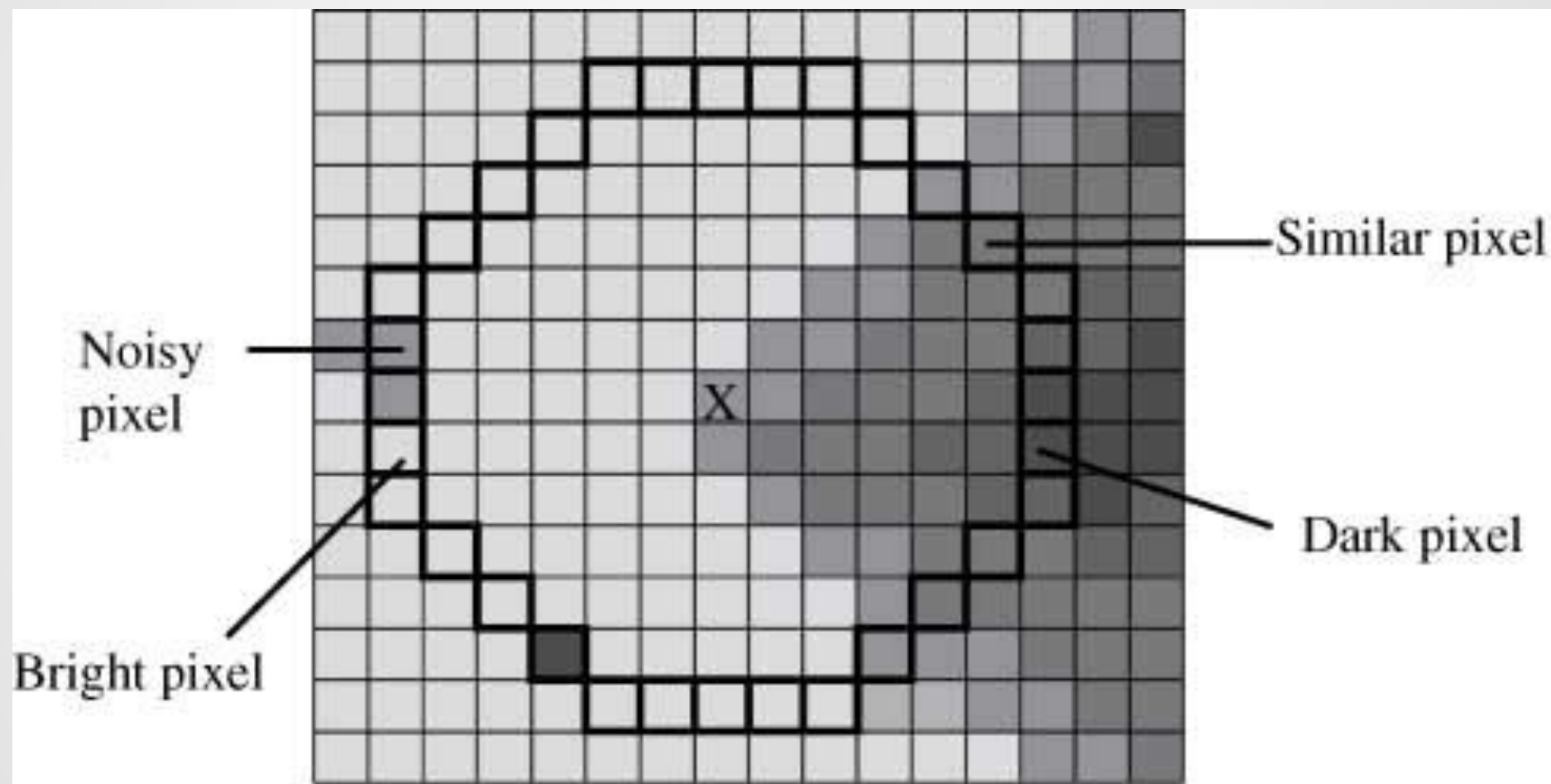
FAST Corner Detector

- **The FAST detector uses a decision tree to identify potential corners**
- **Decision trees are cascaded decision makers**
- **Each one typically makes a yes / no decision using very simple and cheap tests**
- **As a result, FAST is extremely, well, fast**

FAST Corner Detector



FAST Corner Detector



FAST Corner Detector

- Suppose the centre pixel has an intensity value

$$I_p$$

- The pixels are labelled according to

$$I(x, y) = \begin{cases} \textit{bright} & I(x, y) \geq I_p + t \\ \textit{dark} & I(x, y) \leq I_p - t \end{cases}$$

FAST Results (Threshold = 100)



FAST Results (Threshold = 80)



FAST Results (Threshold = 50)



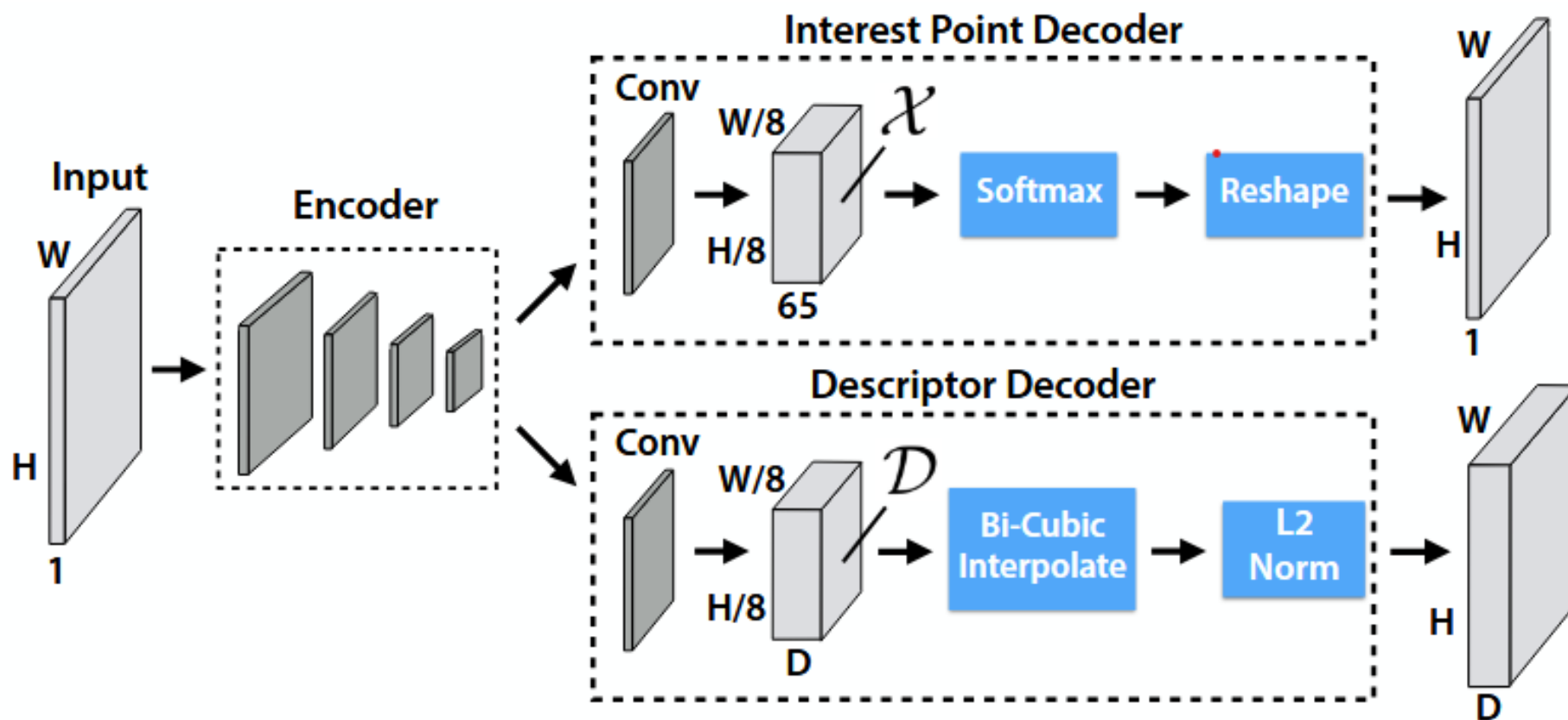
Fast Results (Threshold = 20)



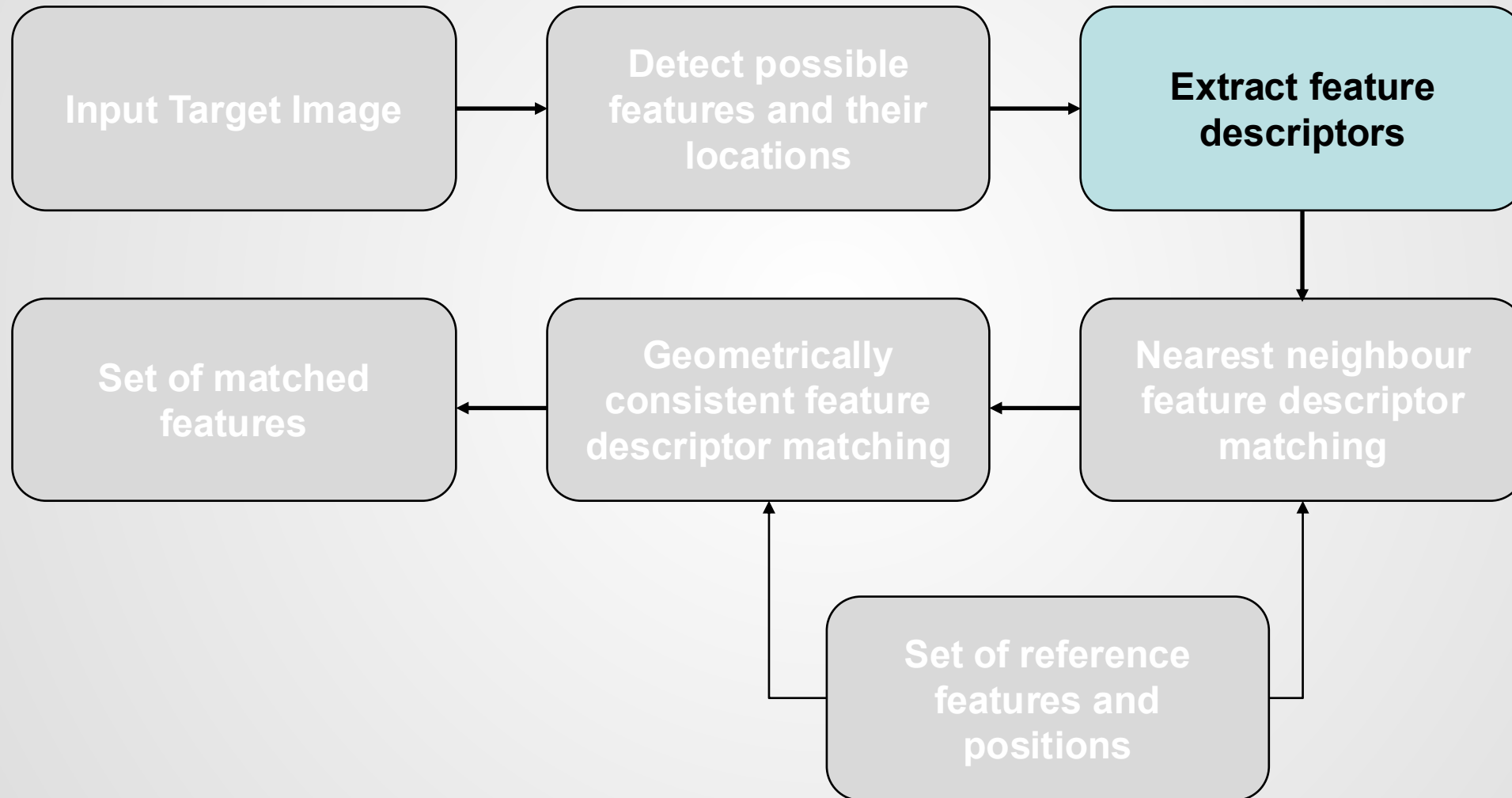
Superpoint

- **Another example of a system is Superpoint**
- **This uses an interest point detector which was jointly trained with the feature descriptor**
- **The system was trained using a combination of synthetic data and applying affine transformations to actual images**

Superpoint Architecture



Extract Feature Descriptors



Characteristics of Feature Descriptors for Landmarks

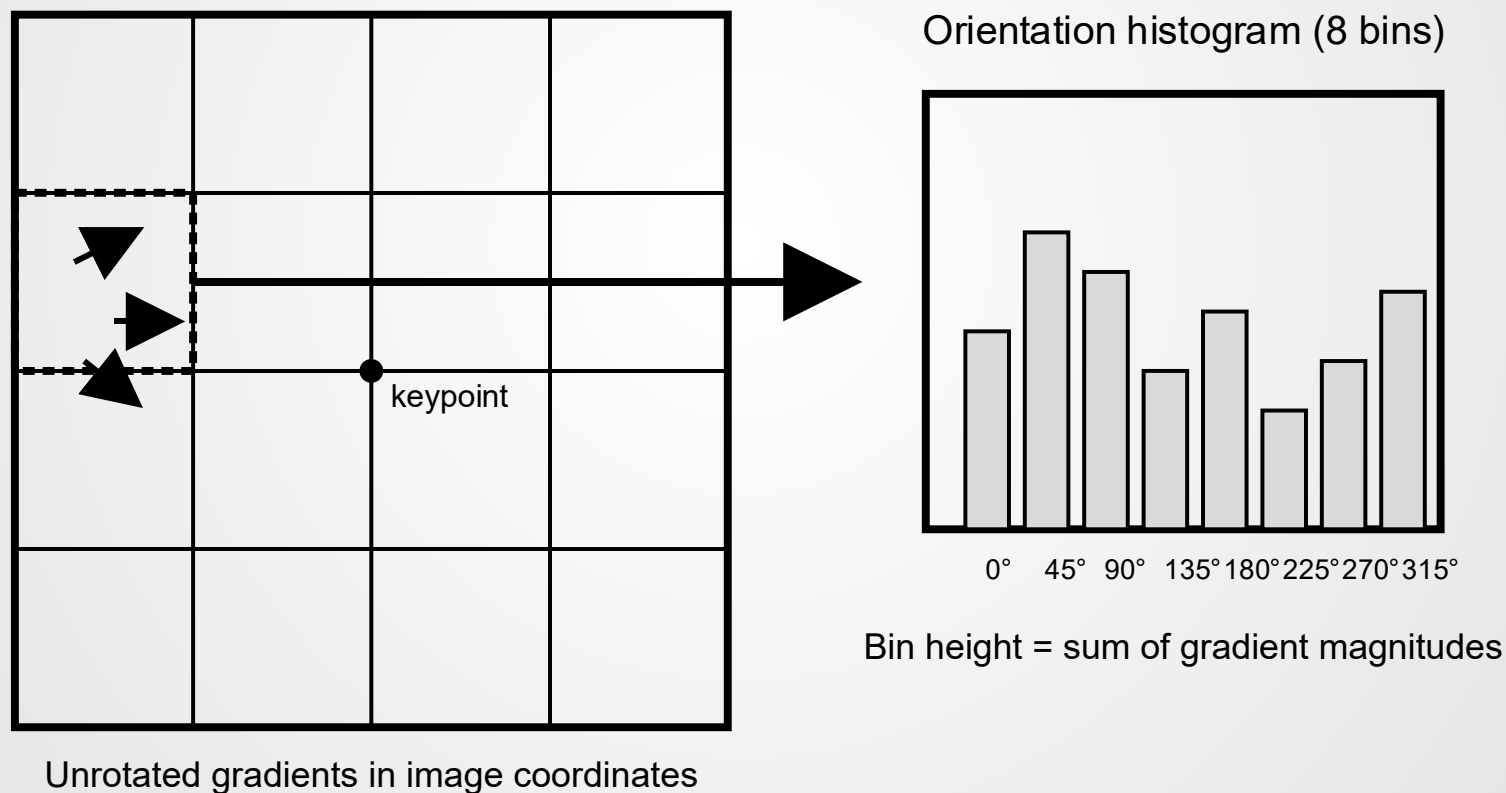
- They must be *distinct* from one another so they can be individually distinguished
- They should be invariant to some degree of matching over translation, rotation and scale
- There are many, many types of feature descriptors
- We'll briefly look at SIFT, ORB and SuperPoint
- All of these use a set of cells around the detected possible feature point

SIFT Feature Descriptors

- SIFT feature descriptors heavily rely on computing histograms of derivatives
- These provide invariance to changes in rotation, scale, etc.
- *An orientation histogram* determines the reference frame

Oriented Histogram from a Single Cell

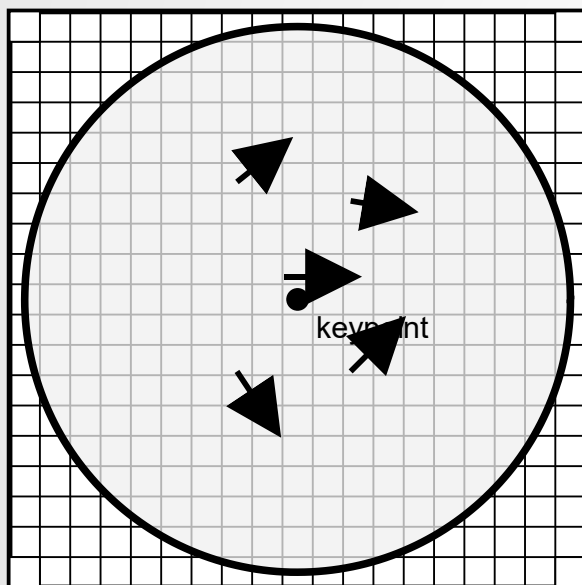
Gradients → Orientation Histogram (single cell)



Orientation Histogram

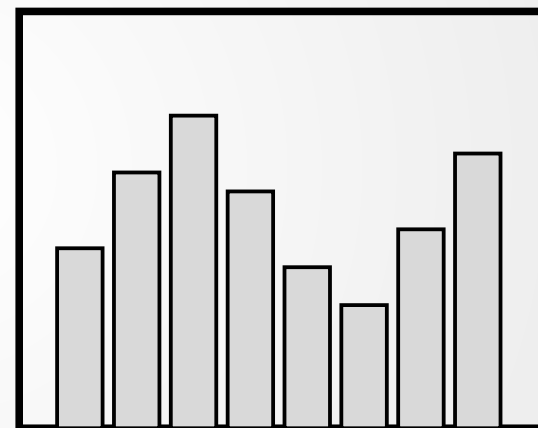
Orientation assignment: circular window shown on a pixel grid

Pixels inside/outside circular neighbourhood



- ☒ inside circular neighbourhood (used)
- ☐ outside neighbourhood (ignored)

Orientation histogram (concept)



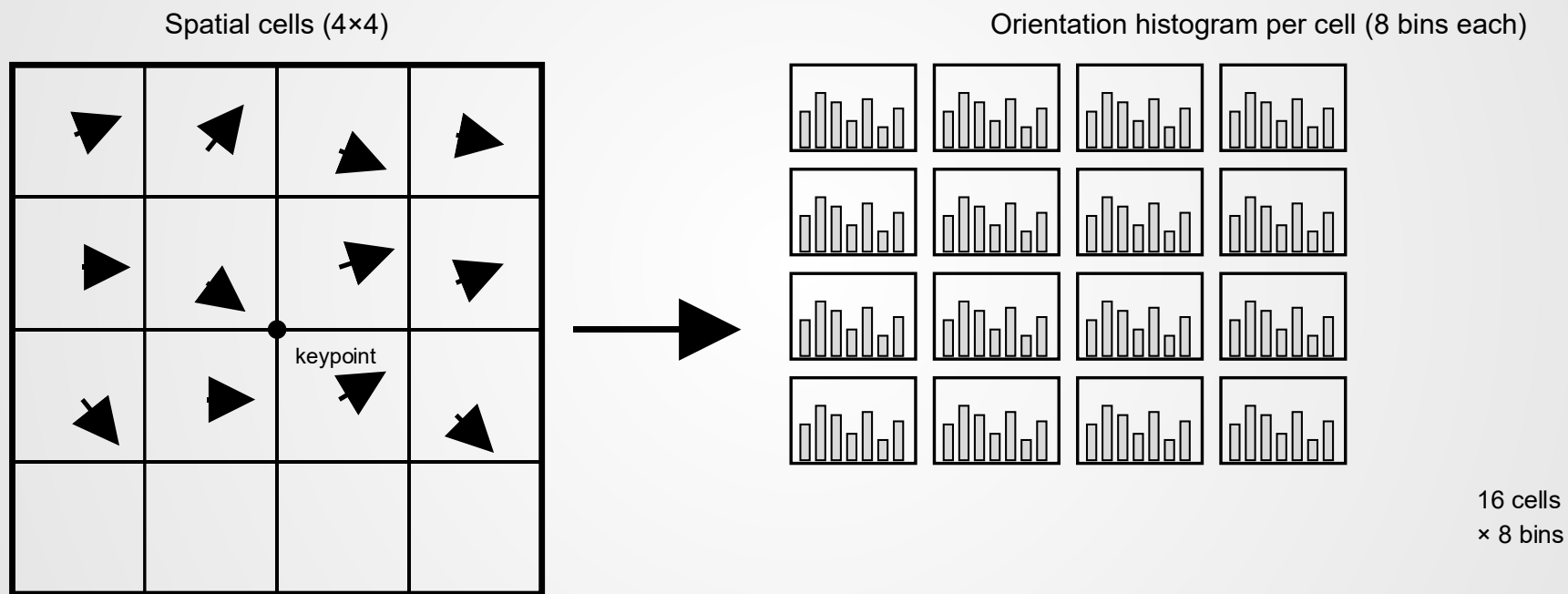
Peak(s) $\rightarrow$ dominant orientation θ_0

SIFT Feature Descriptors

- SIFT feature descriptors heavily rely on computing histograms of derivatives
- These provide invariance to changes in rotation, scale, etc.
- *An orientation histogram* determines the reference frame
- *A descriptor histogram* describes the feature
- Distance is the Euclidean vector

SIFT Descriptor Histogram

SIFT descriptor: 4×4 cells, histogram per cell



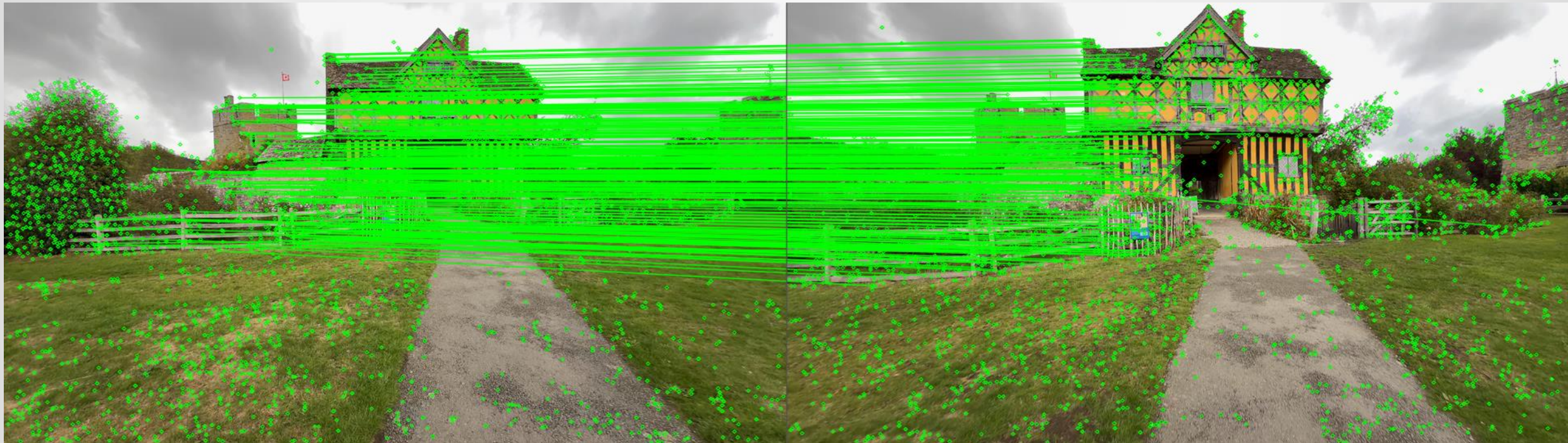
Concatenate all cell histograms → $4 \times 4 \times 8 = 128$ -D descriptor

Normalize → clamp (e.g., 0.2) → renormalize

SIFT Feature Descriptors

- SIFT feature descriptors heavily rely on computing histograms of derivatives
- These provide invariance to changes in rotation, scale, etc.
- *An orientation histogram* determines the reference frame
- *A descriptor histogram* describes the feature
- The distance between the features is just the Euclidean distance

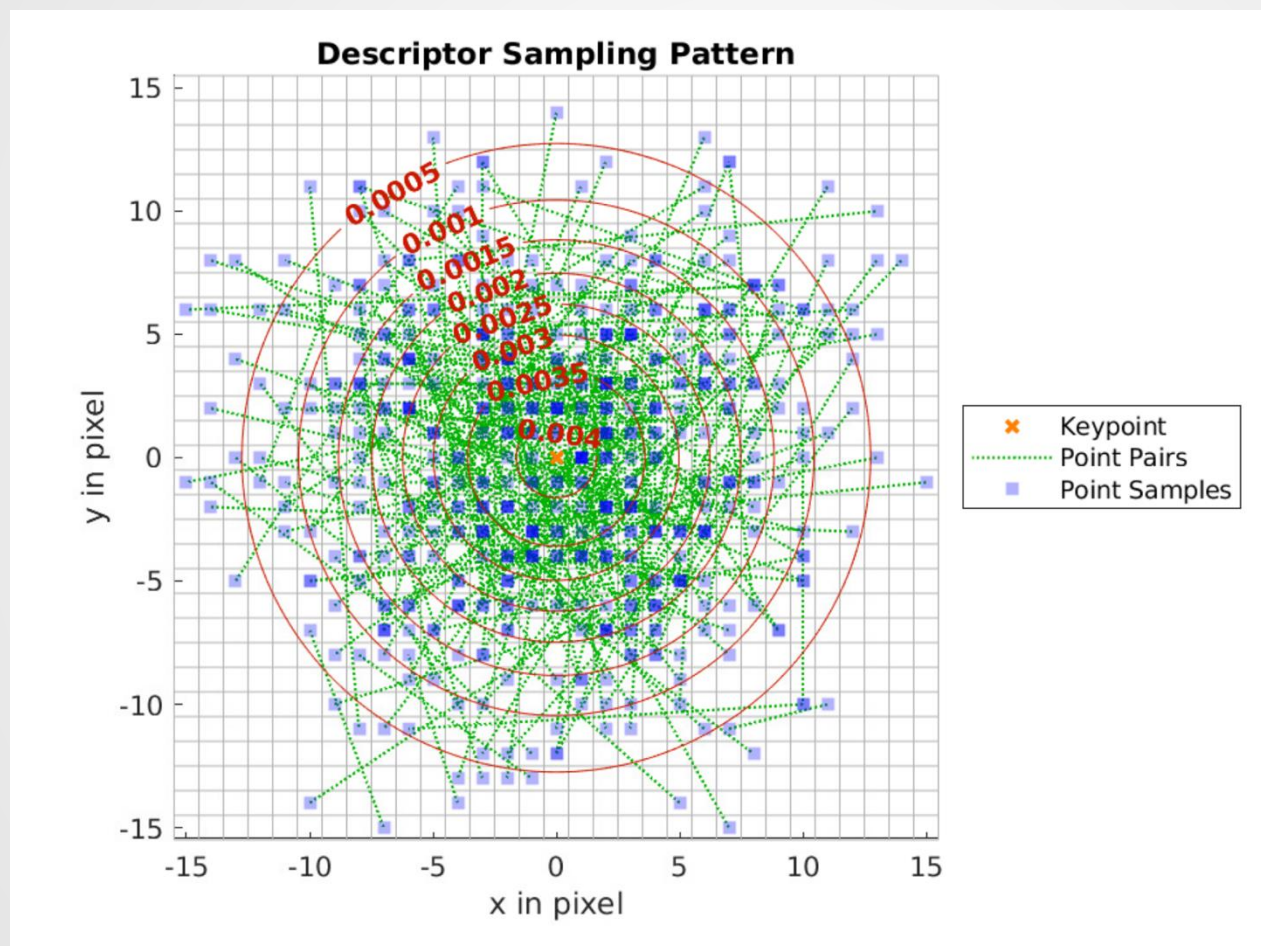
Example of SIFT Matching



ORB Feature Descriptors

- For each interest point identified by FAST, an image pyramid is constructed a bit like SIFT
- Each level is called an image patch
- The moments of the image patch are computed and are used to estimate the centre and rotation of the patch
- To compare ORB features, a random sampling pattern is first created which connects two points

ORB Sampling Pattern



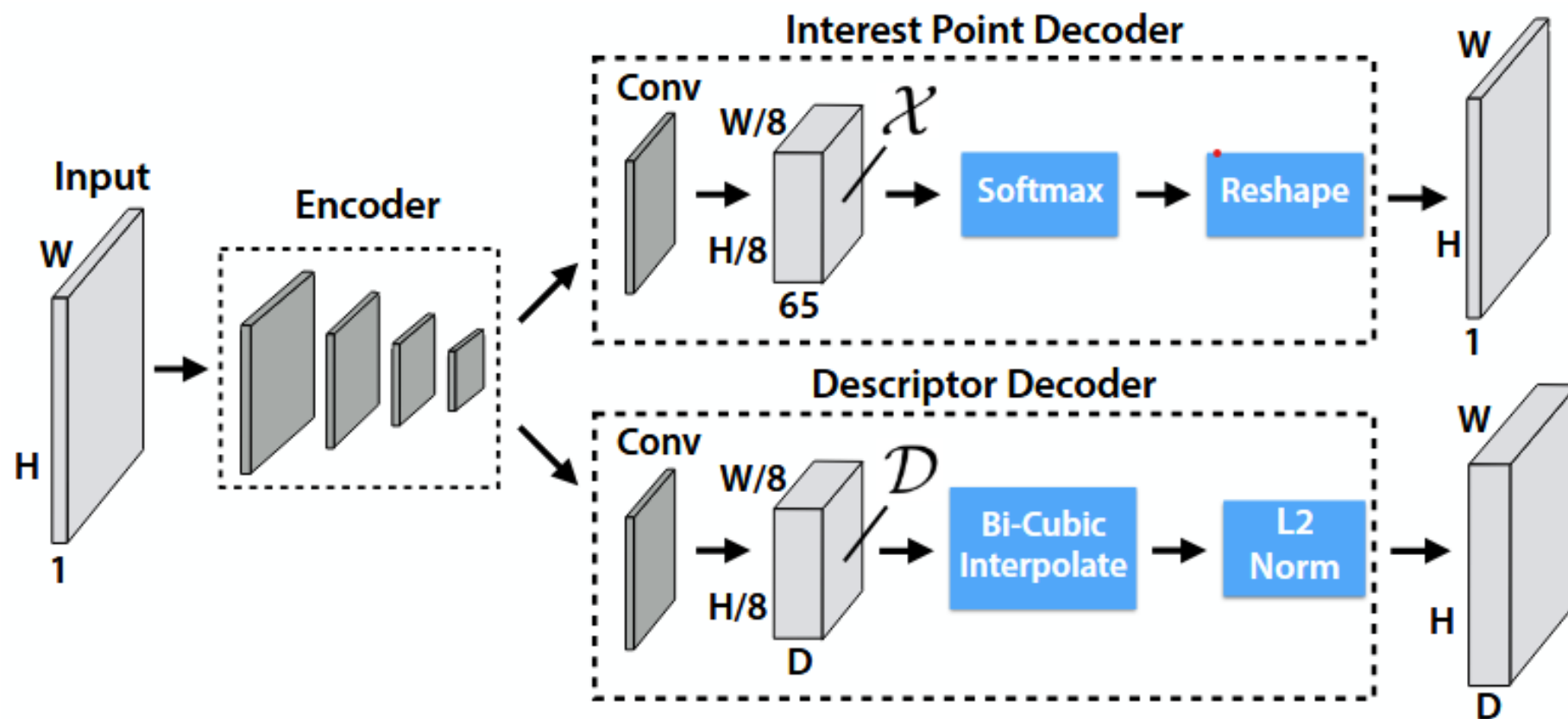
ORB Feature Descriptors

- A binary test is carried out to see which pixel has higher intensity
- This creates a fixed-sized bit string
- Feature distance is the Hamming distance (number of different binary values)

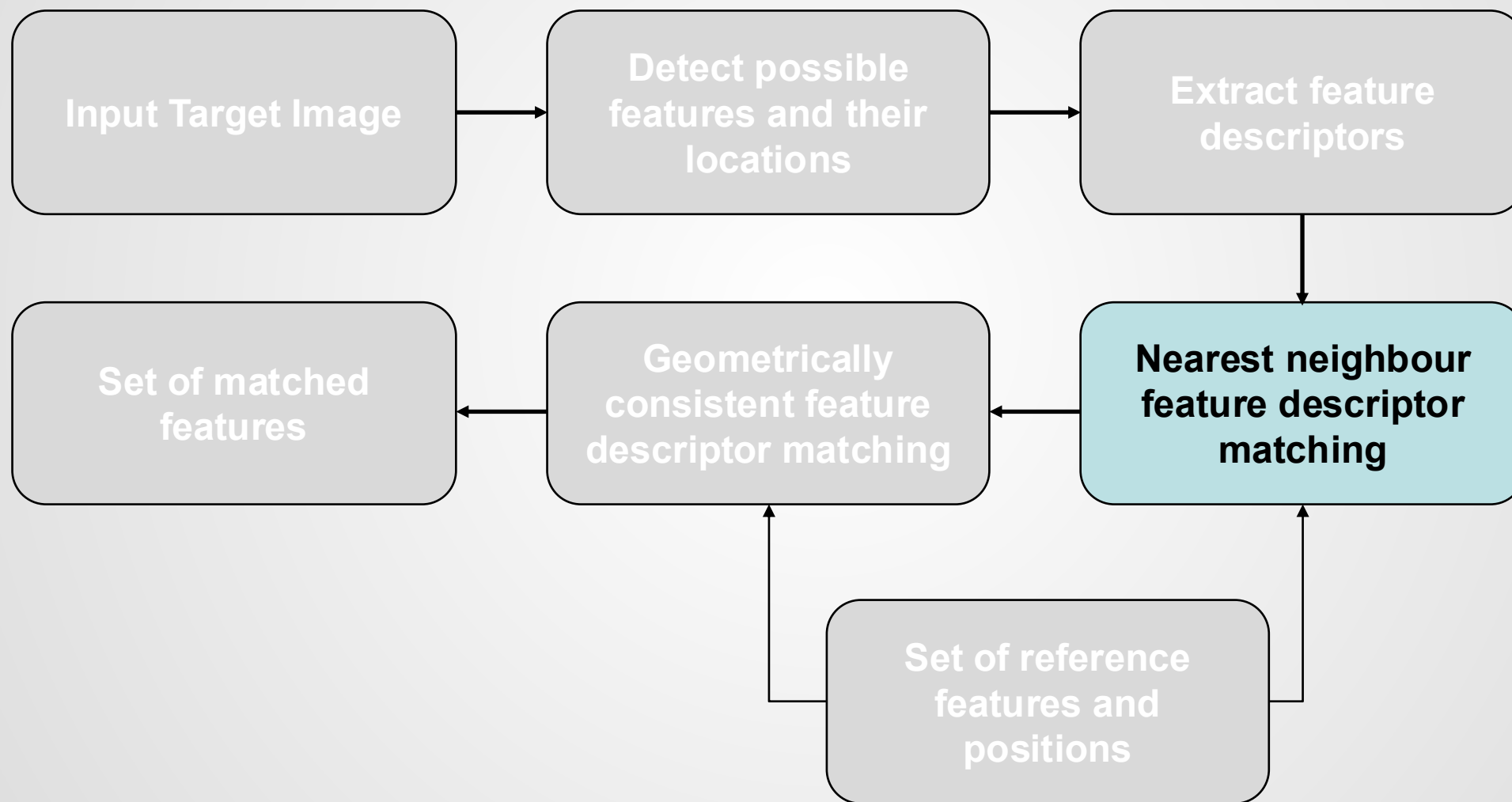
Example of ORB Matching



Superpoint Feature Descriptors



Nearest Neighbour Feature Descriptor Matching



Nearest Neighbour Feature Matching

- **Given a set of extracted keypoints, we now need to match them with our reference set**
- **We assume that for the reference set a similar set of features have been extracted**
- **Therefore, we are trying to do a bijective mapping between the features in the target frame and the reference frame**

Nearest Neighbour Feature Matching



Nearest Neighbour Feature Matching

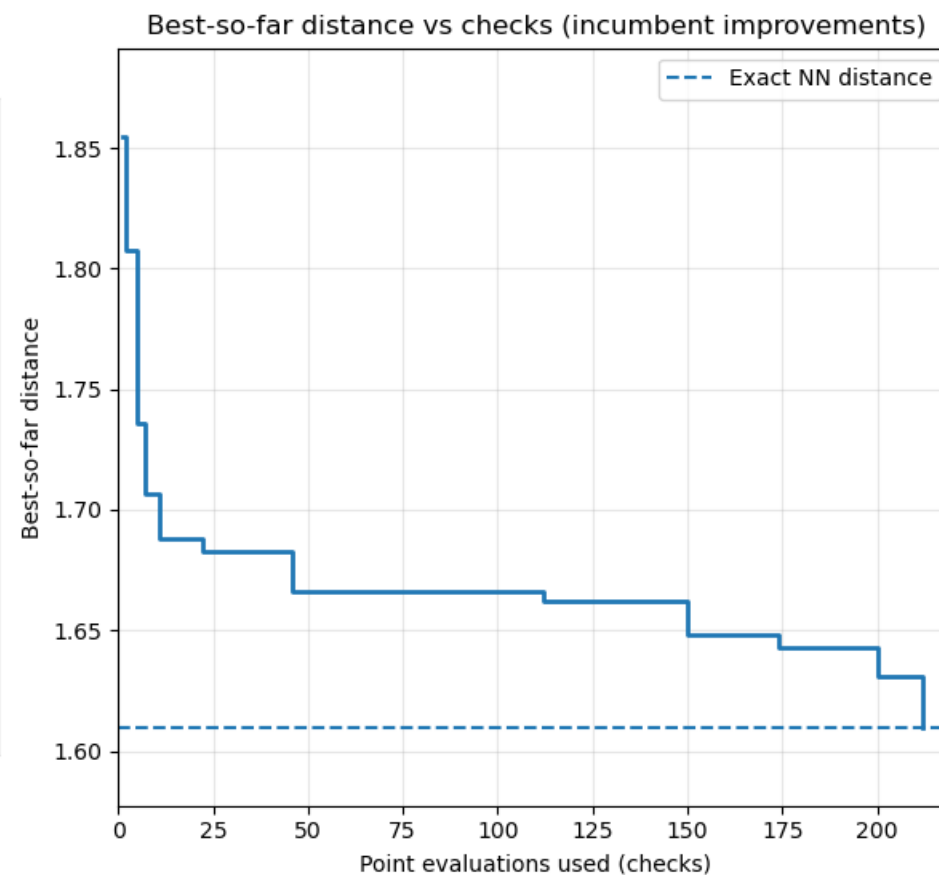
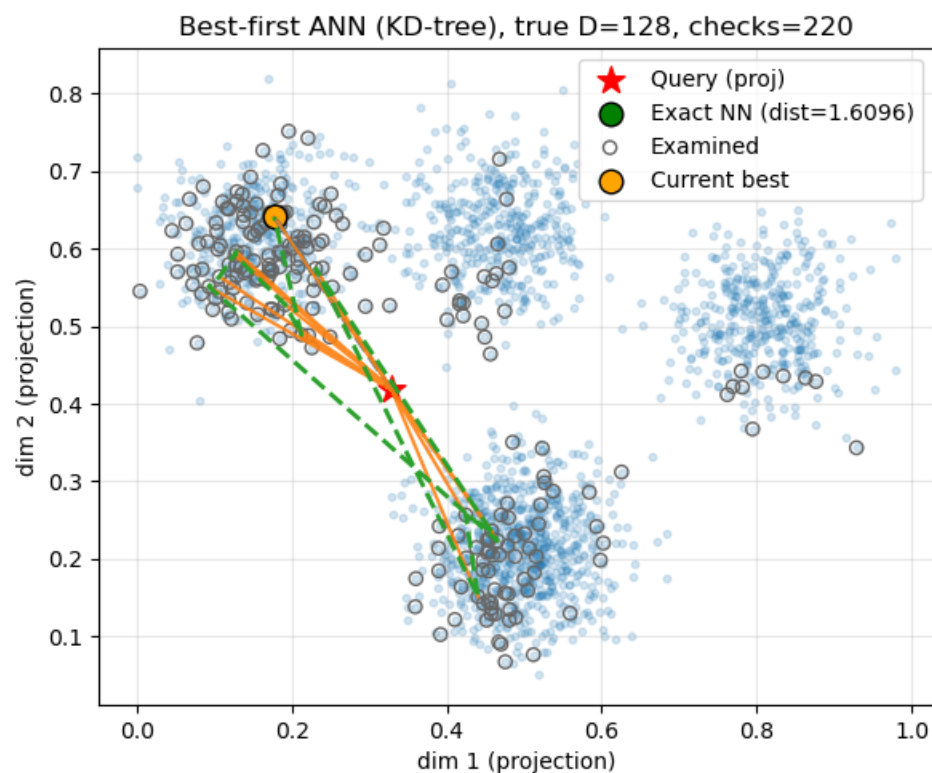
- The challenge is that we want to find the nearest features in very high dimensions
- As a result, pure binary search (which finds the exact match) doesn't work
- A widely used implementation for approximate search is FLANN (Fast Library for Approximate Nearest Neighbors)

FLANN

- A high dimensional search tree is constructed with dimensions equal to those of the feature descriptor
- For each descriptor in the target, we search this structure
- An A*-style search strategy is used, which aggressively searches towards the target but “backs up” if it gets lost
- Search is terminated after a fixed number of steps or if the improvement because too small

Toy Implementation at Work

iter 220/220 | checks used=220/220 | best_dist=1.60955 | exact_dist=1.60955 | ratio=1.000x



Consistent Geometric Transformation



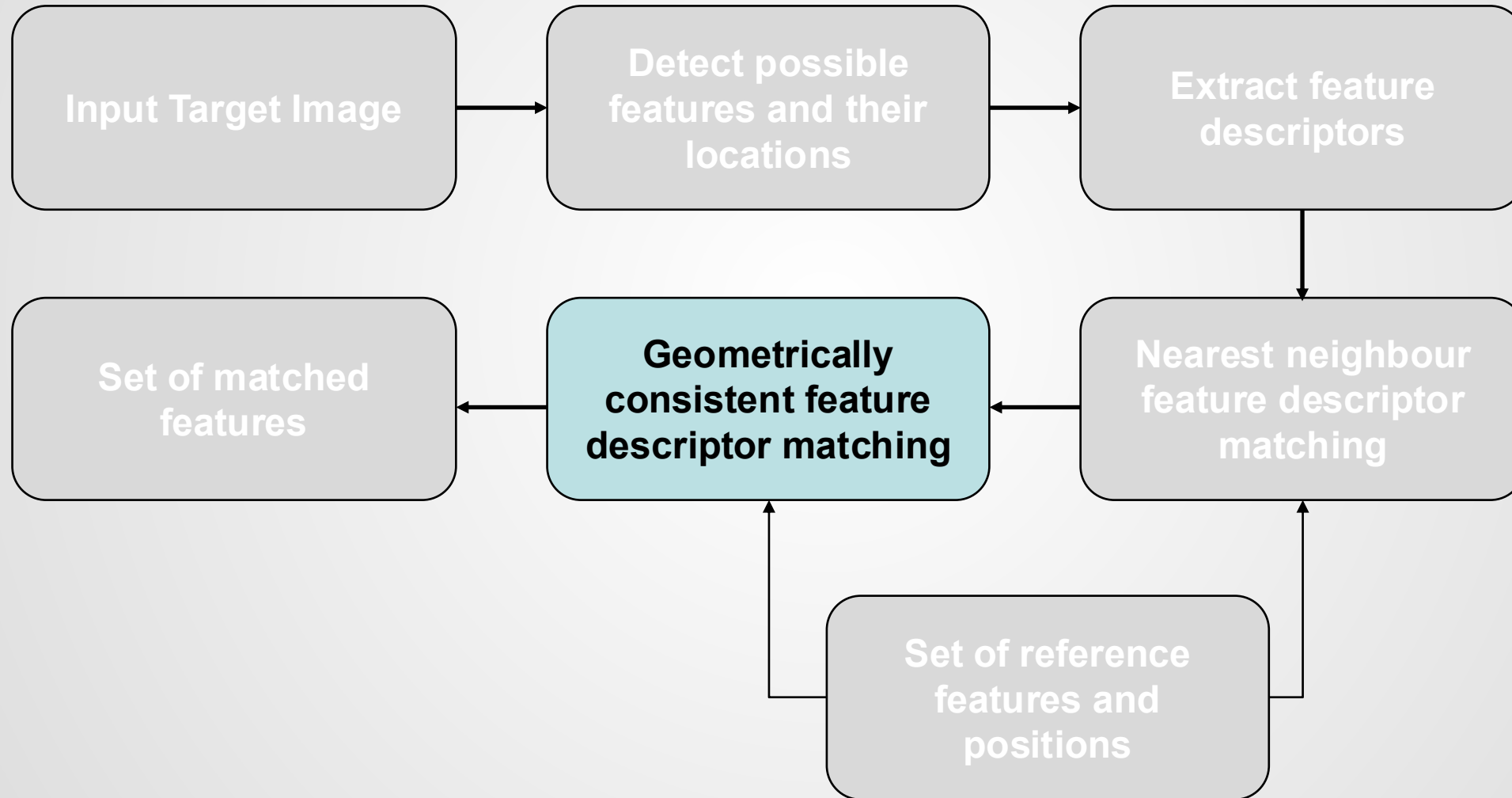
Partially Consistent Geometric Transformation



Inconsistent Geometric Transformation



Nearest Neighbour Feature Descriptor Matching



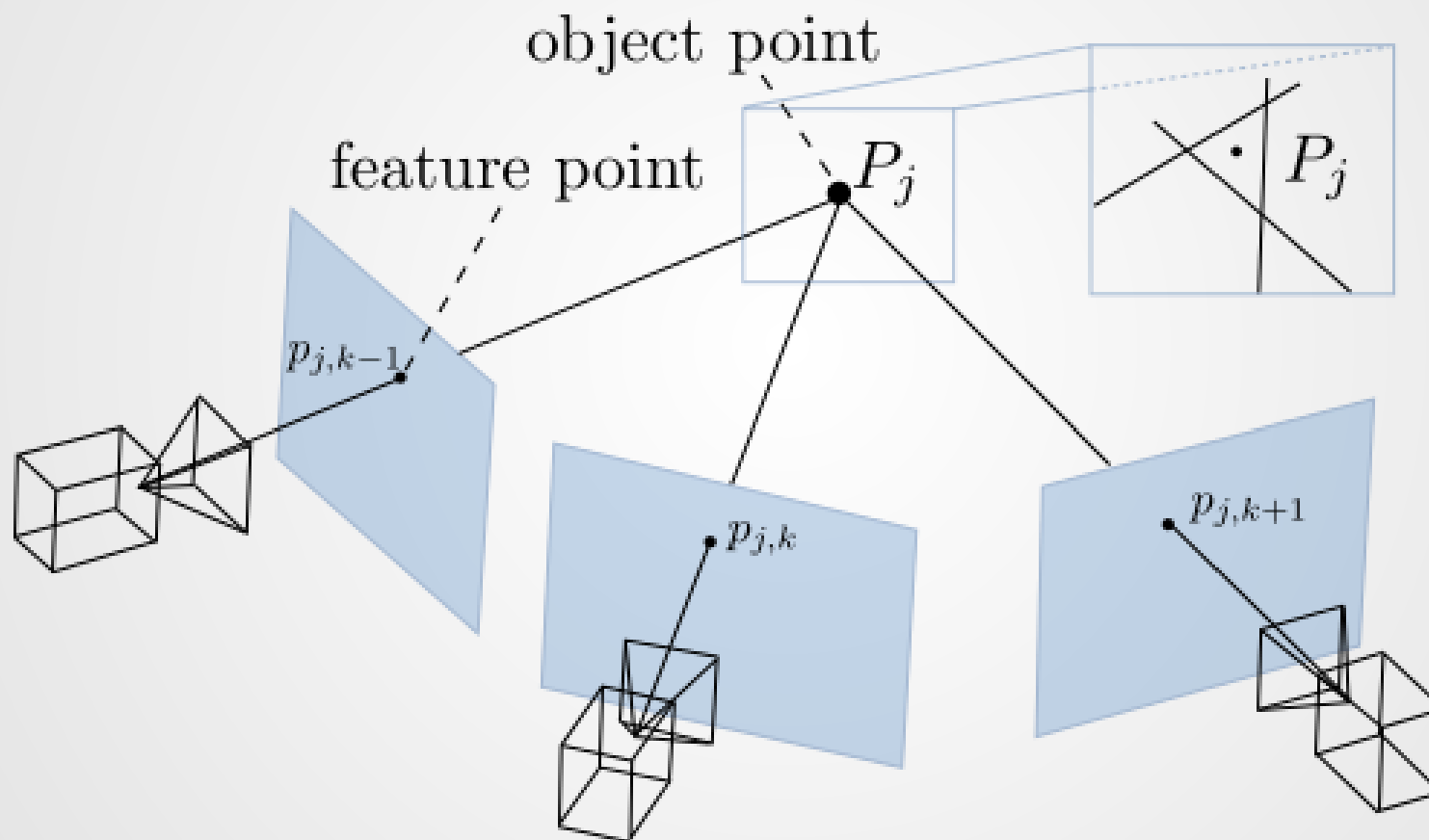
Random Feature Assignment

- **As the last example showed, feature matching just based on visual appearance isn't good enough**
- **Since we don't know which assignments are valid, a common approach to use is Random Consensus Sampling (RANSAC)**
- **The idea is that there are two types of matches:**
 - **Inliers: these are the correct matches that we want to estimate and keep**
 - **Outliers: these are the incorrect matches that we want to discard**

Inliers and Outliers

- **We have to define what inliers and outliers are**
- **We can get them from the setup: a moving camera views a stationary scene:**
 - **Inliers will be all the points whose motion is described by a single camera movement**
 - **Outliers are everything else**
- **We are going to assume we know the 3D positions of the landmarks for now**

Feature Displacement Due to Camera Motion



Feature Displacement Due to Camera and Object Motion



Random Consensus Sampling (RANSAC)

- **The basic idea is:**
 - Pick, at random, n candidate matches and assume they are good
 - Compute the transformation predicted by those candidates
 - Apply the transformation to all the other points in the scene
 - Use a threshold to determine if the other points are inliers or outliers
- **The scorings are normally ranked in terms of inlier count**

Transformations Used

- **When only images are matched:**
 - Homography: requires 4 matches; assumes the scene is largely planar
 - Fundamental matrix; requires 8 matches; does not assume the scene is largely planar
- **When matching with 3D landmark points:**
 - Perspective-n-Point (PnP) problem: 3 matches ambiguous, 4 unambiguous; assumes the points are not colinear

Demonstration....

However, Failure Cases Exist

Summary

- **We've seen how to do frame to frame matching:**
 - Use detectors to work out where feature points might be
 - Use descriptors to describe those points
 - Use FLANN to work out points that look similar
 - Use RANSAC to pick combinations which are geometrically consistent
- **Now let's look at place recognition**

Place Recognition and Loop Closing

- *Frame-to-Frame Feature Matching*
- **Place Recognition and Loop Closing**

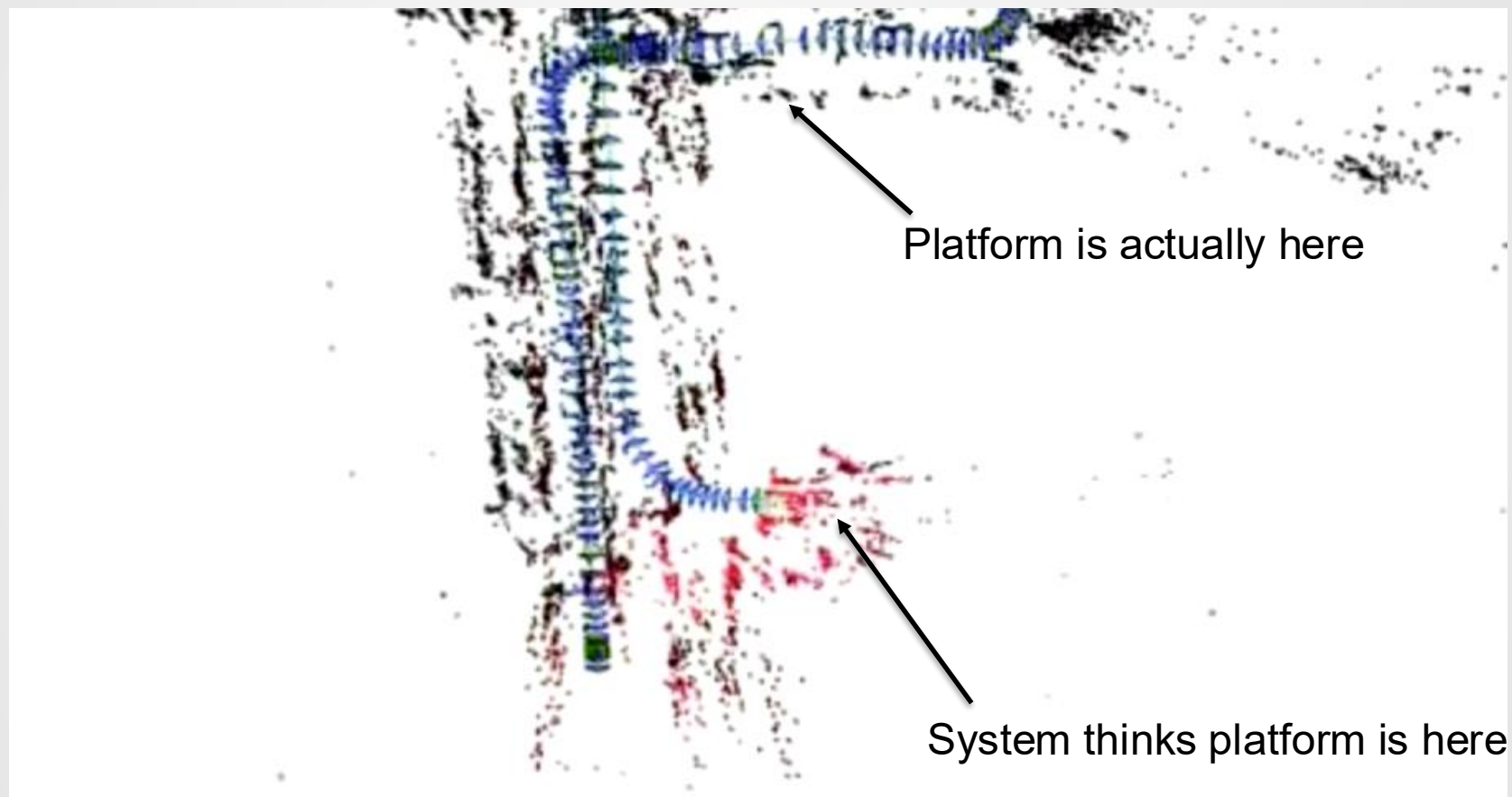
Place Recognition

- **Place recognition answers the questions:**
 - Have I seen this before?
 - If so, where?
- **To carry it out, we have to maintain a history of previous places we have visited**
- **One idea is that we could just store the geometry of the map features and do 3D map matching**

Issues with Geometric Place Recognition

- We'll see later that geometric place recognition can be used with sensors such as LiDAR
- However, for camera-based SLAM systems this is generally a bad idea
- The geometric errors can be very large due to scale drift
- Because the model is sparse, it can be highly ambiguous

Drift in ORB-SLAM

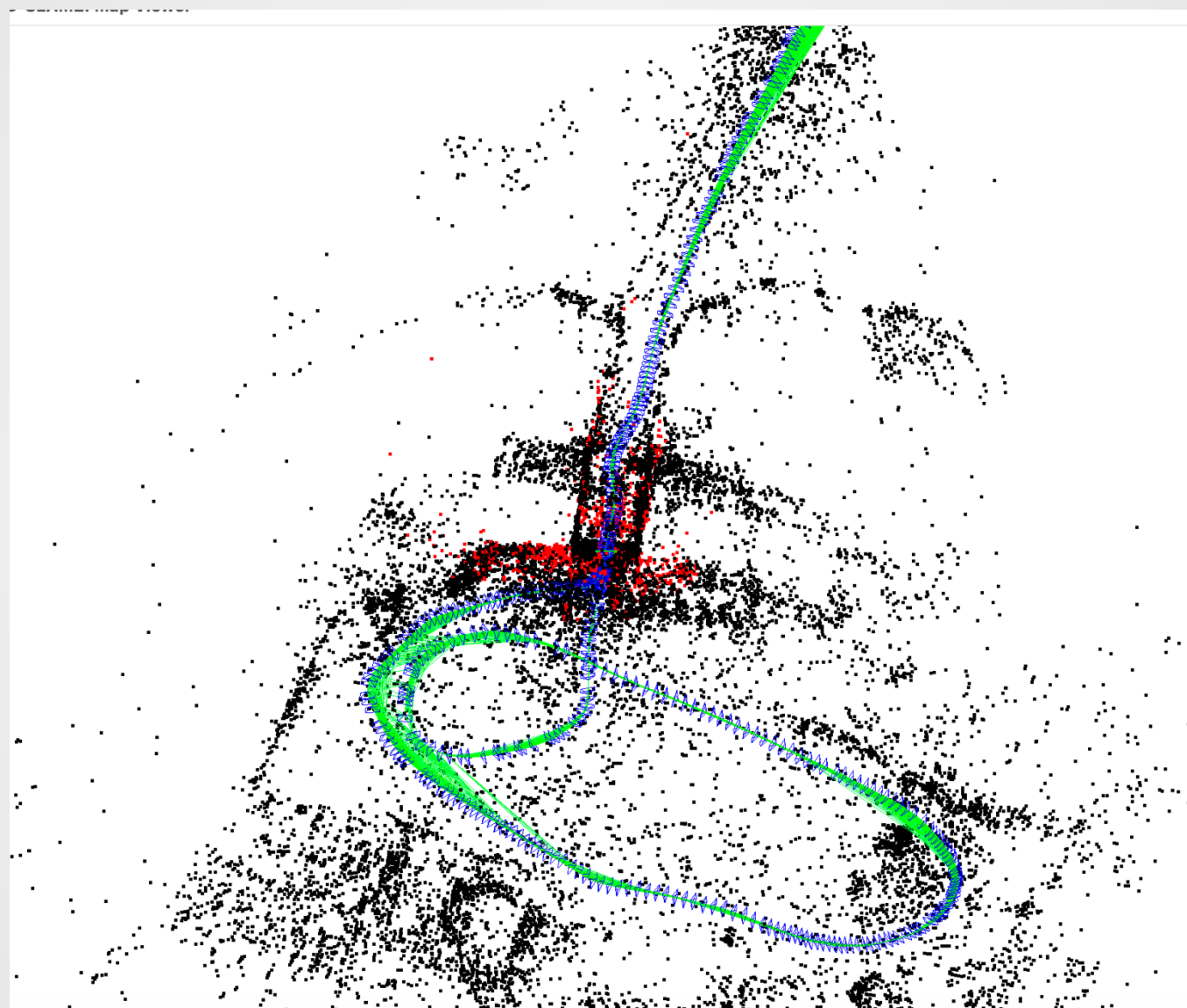


Appearance Place Recognition

- Therefore, place recognition is normally based on appearance
- The idea is to directly compare the input frame with some kind of store of previous image frames
- If we find a match with somewhere we've seen previously, we have identified a candidate loop closure
- What this store looks like differs considerably between SfM and SLAM systems

Image Stores in SfM and SLAM

- In SfM systems such as COLMAP, we store all the frames fed into the system
- In SLAM systems such as ORB-SLAM, we store a subset of frames called *keyframes*
- In the Stokesay dataset:
 - COLMAP stores 3671 frames
 - ORB-SLAM2 stores 498 frames



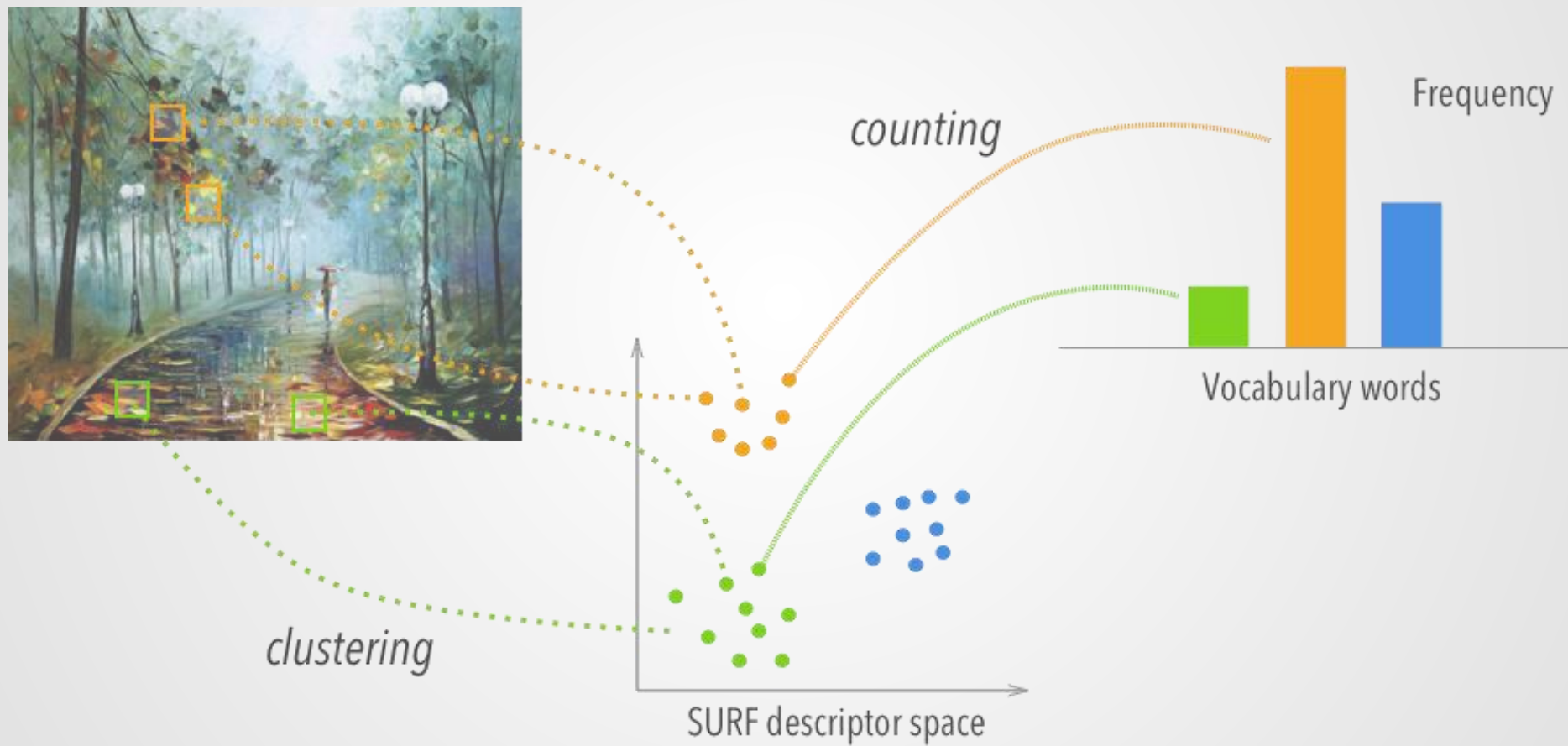
Place Recognition vs. Frame-to-Frame Matching

- Some people argue that place recognition and frame matching are the same thing, and they'll even describe feature matching in tracking as loop closing
- However, we are going to use a distinction:
 - Feature matching is carried out by searching *locally*
 - Place recognition is carried out by searching *globally*

Challenge of Global Search and Feature Dictionaries

- We could apply the frame-to-frame matching for every frame in the database
- However, this will get very slow because potentially we have to check thousands of features in hundreds or thousands of frames
- Therefore, rather than compare each frame directly, an initial faster comparison step is used with a *feature dictionary*
- This approach is known as *Bag Of Words*

Illustrating Bag-of-Words



Building the Dictionary

- The dictionary is built empirically from data
- Feature descriptors are extracted
- Clustering algorithms such as k -means are used
- Feature weighting is applied based on frequency:
 - Very frequent features are given low weight because they are assumed to be not terribly informative
 - Very infrequent features are given a high weight because they are assumed to be informative

Refinements

- **There has been a huge amount of work to build more robust loop closure detection algorithms**
- **Approaches include:**
 - **Use semantics to identify parts of the environment which are likely to be stable**
 - **Use image encoding schemes to produce more robust matching**

Loop Closure

- **Loop closure takes place recognition one step further**
- **Assuming a candidate match has been identified, geometric consistency has to be tested**
- **RANSAC and geometric consistency is used**

Where Next?

- **We've now seen the basic structure of the SLAM estimation problem and where features come from**
- **In the next lecture, we'll look at the architectures of SLAM and SfM systems and see**